

Úvod do jazyka Python

Kontrola dat a databáze

Vladimír Župka 2021, 2022

Obsah

Obsah	2
Použité zdroje informací	4
Stavový stroj (state machine)	5
Kontrola celých čísel	5
Tvar čísla typu int se znaménkem bez podtržítka	5
Hrubý popis činnosti stavového stroje	6
Podrobný popis stavového stroje	7
Funkce se stavovým strojem	8
Příklad	9
Kontrola zlomkových čísel bez exponentu	10
Číslo typu float se znaménkem bez podtržítka	10
Hrubý popis činnosti stavového stroje	11
Podrobný popis stavového stroje	12
Funkce se stavovým strojem	13
Vysvětlivky k funkci	15
Příklad bez chyb	15
Příklad s chybami	15
Kontrola zlomkových čísel s exponentem	16
Hrubý popis činnosti stavového stroje	17
Podrobný popis stavového stroje	18
Číslo s podtržítkem	19
Funkce se stavovým strojem	20
Vysvětlivky k funkci	25
Zkouška chyb	27
Příklad s desítkovými výsledky	28
Příklad s českou lokalizací	29
Kontrola textového vstupu čísel v modulu numpy	30
Databáze SQLite	33
Vytvoření databázových tabulek	33
Dotaz na data	34
Zpracování výsledné tabulky pomocí knihovny Pandas	35
Výstup do formátu HTML	35
Výstup do formátu Excel	36
Výstup do formátu CSV	36
Výstup na konzolu	37
Zápis dat do SQL tabulky z CSV souboru	38
Uzavření přístupu k databázi	39
Databáze MySQL	40
Příprava programovacího prostředí	40
Vytvoření databázových tabulek	41

Dotaz na data	43
Zpracování výsledků dotazu vlastní funkcí	43
Zápis dat do SQL tabulky z CSV souboru.....	44
Uzavření přístupu k databázi	45
Funkce format_result_set().....	45
Databáze PostgreSQL.....	47
Příprava programovacího prostředí	47
Vytvoření databázových tabulek.....	47
Dotaz na data	49
Zpracování výsledků dotazu	49
Zápis dat do SQL tabulky z CSV souboru.....	50
Uzavření přístupu k databázi	50

Použité zdroje informací

Parsers and state machines <https://www.pearsonhighered.com/assets/samplechapter/0/3/2/1/0321112547.pdf>

Python CSV <https://docs.python.org/3/library/csv.html>

Databáze Python sqlite3 <https://docs.python.org/3/library/sqlite3.html>

SQLite <https://sqlite.org/index.html>

Python SQLite <https://zetcode.com/python/sqlite/>

sqlite-utils 3.30, kapitola Python API Introspection <https://sqlite-utils.datasette.io/en/stable/python-api.html>

How to Create and Manipulate SQL Databases with Python <https://www.freecodecamp.org/news/connect-python-with-sql/>

Python MySQL <https://dev.mysql.com/doc/connector-python/en/>

PostgreSQL Python <https://www.postgresqltutorial.com/postgresql-python/>

PostgreSQL driver for Python - psycopg <https://www.psycopg.org>

Kontrola dat

Stavový stroj (state machine)

Stavový stroj (state machine), jiným názvem konečný automat (finite automaton) je programovací metoda založená na rozeznávání stavů při zpracování vstupních dat. Používá se k analýze textu jako lexikální analyzátor (lexical analyzer, scanner) a při určování významu slov jako syntaktický analyzátor (parser). Při analýze textu stavový stroj čte vstupní text znak po znaku, popřípadě ještě s několika znaky dopředu.

Program stavového stroje se zpravidla nachází v několika *stavech* podle složitosti definice správného textu. Má nějaký počáteční stav. Podle toho, jaký znak v daném stavu přečetl, vyhodnotí jej a *přejde* do jiného (nebo zůstane ve stejném) stavu. Vyhodnocení spočívá v tom, zda přečtený znak přijme nebo zda jej zamítne a ohlásí chybu, a do jakého stavu potom přejde. Přijaté znaky může dále zpracovávat. Stav bývá reprezentován nějakým objektem (často číslem).

Na jednoduchém příkladu kontroly celých čísel v *textovém souboru* si ukážeme, jak se stavový stroj programuje.

Kontrola celých čísel

Prvním, snadnějším úkolem bude hledat v textu podřetězce oddělené mezerami (bílémi znaky – whitespace). Zároveň budeme kontrolovat nalezené podřetězce zbavené oddělovacích mezer tak, aby tvořily číslo typu `int` s případným znaménkem `+` nebo `-`. Bude-li podřetězec tvořit číslo, zpracujeme jej tak, že je *zařadíme do seznamu*.

Chybné znaky ohlásíme tiskem a po jejich vynechání se pokusíme získat platné číslo a zařadit je také do seznamu. Výsledkem bude seznam řetězců představujících čísla očištěná od chyb.

Tvar čísla typu `int` se znaménkem bez podtržítka

Formální definice čísla je vyjádřena několika syntaktickými (skladebnými) pravidly nazývanými rozšířená Backus-Naurova forma (BNF). Používá se k popisu bezkontextových gramatik a konečných automatů. U každého pravidla na levé straně od symbolu `::=` je zapsán název syntaktické jednotky, jejíž skladba je popsána na pravé straně pomocí již známých jednotek. Pravidla jsou sestavena tak, aby mohla být seřazena v libovolném pořadí.

```
číslice ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
celky   ::= číslice+
číslo   ::= ['+' | '-'] celky
```

Svislá čárka `|` znamená "nebo". Znaménko `+` za výrazem (`číslice+`) znamená, že výraz musí být přítomen aspoň jednou a může se libovolně opakovat. Výraz v hranatých závorkách `[ ]` je nepovinný.

Regulární výraz této definice vypadá v rozepsaném (verbose) tvaru takto:

```
r'''
[+-]?    # nepovinné znaménko
\d+      # celky
'''
```

Neformálně, číslo *může* mít na začátku jedno znaménko a *musí* obsahovat aspoň jednu číslici.

Takto definovaná čísla budeme hledat v textu, kde mezi čísla je jedna nebo více mezer. Na konci textu nemusí být mezera za číslem. Text může být také prázdný. To je formálně vyjádřeno těmito pravidly:

```
mezera ::= ' ' | '\n' | '\t' | '\r' | '\v' | '\f'
text ::= (mezera* číslo mezera+)* [mezera* číslo]
```

Hvězdička ***** za výrazem (zde **mezera***) znamená, že výraz se může libovolně opakovat a nemusí být vůbec přítomen. Výraz v kulatých závorkách **()** je povinná skupina.

Regulární výraz této definice, v němž symbol **číslo** představuje dříve uvedený regulární výraz, vypadá takto:

```
r'(\s*(číslo)\s+)*(\s*(číslo))?'
```

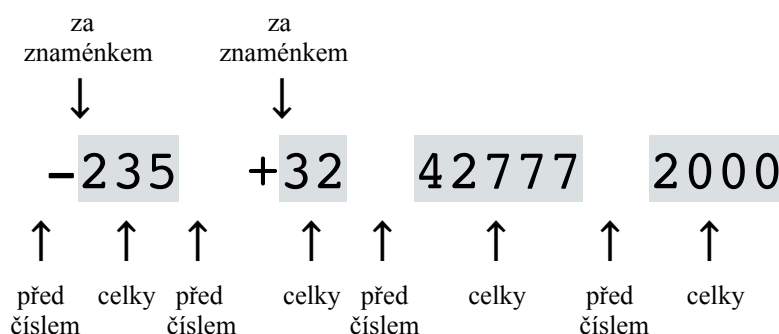
Hrubý popis činnosti stavového stroje

Jak snad uznáme, syntaktická pravidla jsou názornější než regulární výrazy. Ale ať už se budeme řídit syntaktickými pravidly nebo regulárním výrazem, zpracování proběhne v cyklu tak, že z řetězce nebo souboru se čte *znak po znaku*. Jaké stavy budou nastávat při zpracování znaků, nevyplývá z formálních definic přímo. Několikerou úvahou můžeme dospět k tomu, že program bude rozeznávat následující stavy:

PŘED_ČÍSLEM na začátku nebo když nebylo přečteno znaménko ani číslice
 ZA_ZNAMENKEM bylo přečteno znaménko
 CELKY byla přečtena číslice
 konec vstupu

Počáteční stav bude PŘED_ČÍSLEM. V každém stavu program vyhodnotí a zpracuje přečtený znak, načež přečte další znak a podle něj přejde do jiného stavu nebo zůstane ve stejném stavu. Při vyhodnocování povolený znak zařadí do čísla, nepovolený znak ohlásí jako chybu; mezeru a nepovolený znak vynechá. Vždy, když se ve stavu CELKY přečte mezera nebo je rozeznán konec dat, shromážděné povolené znaky se zařadí do seznamu.

Příklad textu



Podrobný popis stavového stroje

Začíná stavem PŘED_ČÍSLEM a čte první znak.

Když byl ve stavu

PŘED_ČÍSLEM přečten znak

mezera, přečte další znak a přejde do stavu PŘED_ČÍSLEM

+ nebo **-**, zařadí znak do čísla, přečte další znak a přejde do stavu ZA_ZNAMENKEM

číslice, zařadí znak do čísla, přečte další znak a přejde do stavu CELKY

jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu PŘED_ČÍSLEM

ZA_ZNAMENKEM přečten znak

číslice, zařadí znak do čísla, přečte další znak a přejde do stavu CELKY

jiný znak, ohlásí chybu, přečte další znak a přejde do stavu PŘED_ČÍSLEM

CELKY přečten znak

číslice, zařadí znak do čísla, přečte další znak a zůstane ve stavu CELKY

mezera, přečte další znak, *zpracuje* číslo a přejde do stavu PŘED_ČÍSLEM

jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu CELKY

Když *na konci souboru* cyklus skončí ve stavu CELKY, tj. když chyběla poslední mezera, zpracuje se ještě poslední číslo. Zpracování spočívá v zápisu čísla do seznamu.

Funkce se stavovým strojem

```
def cela_cisla(file):
    """ Funkce zjišťuje celá čísla ve znakovém tvaru oddělená mezerami, hlásí chyby.
    Vrací seznam očištěných čísel ve znakovém tvaru.
    Parametr `file` je otevřený textový soubor.
    """
    scisla = [] # seznam očištěných čísel
    cislo = '' # řetězec pro střeďání znaků čísla
    PRED_CISLEM = 1
    ZA_ZNAMENKEM = 2
    CELKY = 3
    znak = ''

    # Funkce 'dalsi()' přečte jeden další znak ze souboru
    def dalsi():
        nonlocal znak
        znak = file.read(1)

    # Tělo hlavní funkce

    dalsi() # čtu první znak
    if znak == '': # konec souboru, vrátím prázdný seznam
        return scisla

    stav = PRED_CISLEM # počáteční stav
    print(f"Čtené znaky:")

    # Hlavní cyklus
    while True:
        if znak == '': # prázdný znak značí konec souboru
            print(f"{repr(znak)}konec dat.")
            break # ukončím cyklus
        print(znak, end='')
        if stav == PRED_CISLEM:
            cislo = '' # vyprázdním řetězec pro střeďání znaků čísla
            if znak.isspace():
                dalsi(); continue
            elif znak == '+' or znak == '-': # znaménko?
                cislo += znak
                stav = ZA_ZNAMENKEM; dalsi(); continue
            elif znak.isdigit(): # číslice?
                cislo += znak
                stav = CELKY; dalsi(); continue
            else:
                print(f"\n          chybný znak {repr(znak)}"
                      f" ve stavu PRED_CISLEM")
                dalsi(); continue

        elif stav == ZA_ZNAMENKEM:
            if znak.isdigit(): # číslice?
                cislo += znak
                stav = CELKY; dalsi(); continue
            else:
                print(f"\n          chybný znak {repr(znak)}"
                      f" ve stavu ZA_ZNAMENKEM")
                stav = PRED_CISLEM; dalsi(); continue

        elif stav == CELKY:
            if znak.isdigit(): # číslice?
                cislo += znak
                dalsi(); continue
            elif znak.isspace(): # mezera?
                scisla.append(cislo) # zpracuji číslo
                stav = PRED_CISLEM; dalsi(); continue
```



```

else:
    print(f"\n                chybný znak {repr(znak)}"
          f" ve stavu CELKY")
    dalsi(); continue

# Konec hlavního cyklu:
if stav == CELKY: # chybí mezera za posledním číslem
    scisla.append(cislo) # zpracuji poslední číslo
return scisla

```

Příklad

V příkladu nejprve zapíšeme kontrolovaný text do souboru a pak stejný soubor kontrolujeme jako vstupní. Ve vstupním textu jsou záměrně chyby, které funkce zachytí, oznámí, odstraní a pokračuje v kontrole dále až do konce. Při tom všechny přečtené znaky tiskne a zapisuje do výstupního souboru. Očištěné řetězce zapisuje do seznamu a nakonec seznam vytiskne.

Příprava vstupních dat

```

# zápis zkušebních dat do souboru
outfile = open("cisla.txt", "w")
outfile.write("-235 + 32 42777 2000 *19 6 42")
outfile.close()

```

Výstupní a pak vstupní soubor

```
-235 + 32 42777 2000 *19 6 42
```

Spuštění funkce

```

# otevření vstupního souboru čísel
infile = open("cisla.txt", "r")
cisla = cela_cisla(infile) # vrací seznam řetězců

```

Tisk mezivýsledků

```

-235 +
                chybný znak ' ' ve stavu ZA_ZNAMENKEM
32 42777 2000 *
                chybný znak '*' ve stavu PRED_CISLEM
19 6 42' konec dat.

```

Tisk výsledku

```

print("Očištěná čísla:\n", cisla) # tiskne vrácený seznam
Očištěná čísla:
['-235', '32', '42777', '2000', '19', '6', '42']

infile.close() # nepovinně uzavřu soubor

```

Kontrola zlomkových čísel bez exponentu

Dalším úkolem bude hledat v *textovém souboru* podřetězce oddělené *právě jednou čárkou* a možnými mezerami (bílémi znaky – whitespaces). Zároveň budeme kontrolovat nalezené podřetězce zbavené oddělovačů tak, aby tvořily celé číslo nebo *zlomkové číslo* typu `float` s případným znaménkem podle formální definice, zatím *bez exponentu* (např. 1.234e5). Tento úkol je již těžší, komplikace přináší desetinná tečka a oddělovací čárka.

Chybné znaky ohlásíme tiskem a po jejich vynechání se pokusíme získat platné *číslo* a *zpracovat* je pro výpočet maxima, minima a průměru. Tyto hodnoty na konci vytiskneme.

Číslo typu `float` se znaménkem bez podtržítka

Formální definice zlomkového čísla je vyjádřena následujícími syntaktickými pravidly.

```
číslice      ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
celky        ::= číslice+
zlomek       ::= '.' celky
zlomkové číslo ::= [celky]* zlomek
celky nebo zlomek ::= celky ['.' ]
číslo        ::= ['+' | '-'] (zlomkové číslo | celky nebo zlomek)
```

Svislá čárka `|` znamená "nebo". Znaménko `+` za výrazem (`číslice+`) znamená, že výraz musí být přítomen aspoň jednou a může se libovolně opakovat. Hvězdička `*` za výrazem znamená, že výraz se může libovolně opakovat, ale nemusí být vůbec přítomen. Výraz v hranatých závorkách `[ ]` je nepovinný. Výraz v kulatých závorkách `( )` je povinná skupina. Některé značky se shodují se značkami používanými v regulárních výrazech (plus, hvězdička, svislá čárka, kulaté závorky) a některé se používají jinak (zdejší hranaté závorky jsou v regulárním výrazu vyjádřeny otazníkem).

Regulární výraz této definice může vypadat v rozepsaném (verbose) tvaru takto:

```
r'['[+-]? # nepovinné znaménko
(
  \d*    # nepovinné celky
  [.]    # tečka
  \d+    # celky
)
|
(
  \d+    # celky
  [.]?   # nepovinná tečka
)'
```

Neformálně, číslo bez znaménka *musí* obsahovat aspoň jednu číslici, *může* mít tečku uvnitř, na začátku nebo na konci (nebo nikde) např.: 01.10 .10 01. 01

Takto definovaná čísla budeme hledat v textu, kde mezi čísly je jedna povinná čárka a libovolný (i nulový) počet mezer. Na konci textu nemusí být za číslem čárka. Text může být také prázdný. To je formálně vyjádřeno těmito pravidly:

```
whitespace ::= ' ' | '\n' | '\t' | '\r' | '\v' | '\f'
mezery     ::= whitespace*
text       ::= ([mezery] číslo [mezery] ',')* ([mezery] číslo [mezery])
```

Regulární výraz této definice, v němž symbol *číslo* představuje dříve uvedený regulární výraz, vypadá takto:

```
r'(\s*(číslo)\s*[,])*(\s*(číslo)\s*)?'
```

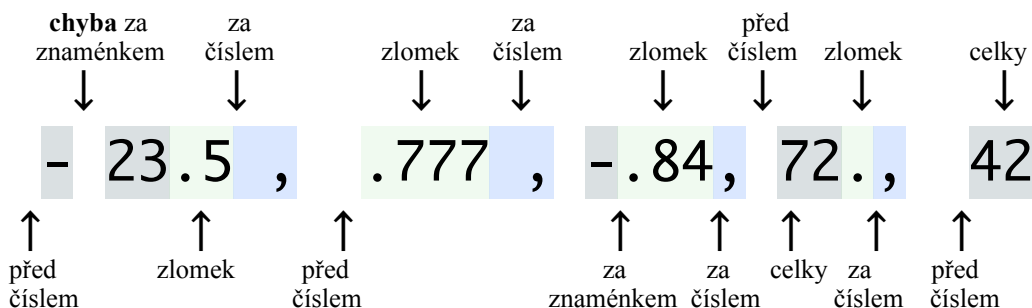
Hrubý popis činnosti stavového stroje

Ať už se budeme řídit syntaktickými pravidly nebo regulárním výrazem, zpracování proběhne v cyklu tak, že z řetězce nebo souboru se čte znak po znaku. Program přitom bude rozeznávat následující stavy:

PŘED_ČÍSLEM na začátku a po přečtení čárky
ZA_ZNAMENKEM bylo přečteno znaménko
CELKY byla přečtena číslice
ZLOMEK byla přečtena tečka
ZA_ČÍSLEM byla přečtena mezera
konec vstupu

V každém stavu program vyhodnotí a zpracuje přečtený znak, načež přečte další znak a podle něj přejde do stejného nebo jiného stavu. Při vyhodnocování povolený znak zařadí do čísla, nepovolený znak ohlásí jako chybu, mezeru (whitespace), čárku a nepovolený znak vynechá. Vždy, když se přečte čárka, shromážděné povolené znaky se zpracují jako číslo. Číslo se zpracuje také tehdy, když za ním *na konci textu* nebyla přečtena čárka.

Příklad textu



Podrobný popis stavového stroje

Začíná stavem PRED_CISLEM a čte první znak.

Když byl ve stavu

PRED_CISLEM přečten znak

mezera, přečte další znak a zůstane ve stavu PRED_CISLEM

+ nebo **-**, zařadí znak do čísla, přečte další znak a přejde do stavu ZA_ZNAMENKEM

tečka, zařadí znak do čísla, přečte další znak a přejde do stavu ZLOMEK

číslice, zařadí znak do čísla, přečte další znak a přejde do stavu CELKY

jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu PRED_CISLEM

ZA_ZNAMENKEM přečten znak

číslice, zařadí znak do čísla, přečte další znak a přejde do stavu CELKY

tečka, zařadí znak do čísla, přečte další znak a přejde do stavu ZLOMEK

jiný znak, ohlásí chybu, přečte další znak a přejde do stavu PRED_CISLEM

CELKY přečten znak

číslice, zařadí znak do čísla, přečte další znak a zůstane ve stavu CELKY

tečka, zařadí znak do čísla, přečte další znak a přejde do stavu ZLOMEK

mezera, přečte další znak a přejde do stavu ZA_CISLEM

čárka, *zpracuje* číslo, přečte další znak a přejde do stavu PRED_CISLEM

jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu CELKY

ZLOMEK přečten znak

číslice, zařadí znak do čísla, přečte další znak a zůstane ve stavu ZLOMEK

mezera, přečte další znak a přejde do stavu ZA_CISLEM

čárka, *zpracuje* číslo, přečte další znak a přejde do stavu PRED_CISLEM

jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu ZLOMEK

ZA_CISLEM přečten znak

mezera, přečte další znak a přejde do stavu ZA_CISLEM

čárka, *zpracuje* číslo, přečte další znak a přejde do stavu PRED_CISLEM

jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu ZA_CISLEM

Když *cyklus skončí* v jiném stavu než PRED_CISLEM, tedy, když chyběla poslední čárka, *zpracuje* se ještě poslední číslo.

Zpracování spočívá v dokončení čísla sestaveného z povolených čtených znaků:

- kontrola *úplnosti* čísla (zda nechybí číslice),
- zařazení čísla do seznamu,
- převod do čísla typu `float`,
- výpočet průběžného maxima, minima, součtu a počtu čísel.

Funkce se stavovým strojem

```
def císla_bez_exponentu(file):
    """*** Funkce zjišťuje čísla ve znakovém tvaru a počítá z nich
    maximum, minimum, průměr."""
    scísla = [] # seznam očištěných čísel ve znakovém tvaru
    maximum = float('-inf')
    minimum = float('inf')
    soucet = 0.0
    pocet = 0
    cislo = '' # řetězec pro střeádání znaků čísla
    PRED_CISLEM = 1
    ZA_ZNAMENKEM = 2
    CELKY = 3
    ZLOMEK = 4
    ZA_CISLEM = 5

    # Funkce pro výpočet souhrnů max, min, sum, count
    def zpracovani():
        nonlocal maximum, minimum, soucet, pocet
        if any(c.isdigit() for c in cislo):
            scísla.append(cislo)
            fl = float(cislo)
            if fl > maximum:
                maximum = fl
            if fl < minimum:
                minimum = fl
            soucet += fl
            pocet += 1
        else:
            print(f"\n      neúplné {pocet+1}. čísla: {repr(cislo)}")

    # Tělo hlavní funkce

    znak = file.read(1) # číst první znak
    if znak == '': # když je to prázdný znak - "konec souboru", vrátit "Prázdný vstup"
        return {"maximum": "? - prázdný vstup",
                "minimum": "? - prázdný vstup", "prumer": "? - prázdný vstup"}
    file.seek(0) # když je to neprázdný znak, vrátit na začátek souboru

    stav = PRED_CISLEM
    znak = file.read(1) # čtu první znak
    print("Čtené znaky: ", end='')

    # Hlavní cyklus
    while True:
        if znak == '': # prázdný znak značí konec souboru
            break # už ho nezpracuji, aby se na něm nehlásila chyba
        print(znak, end='') # otisk čteného znaku
        if stav == PRED_CISLEM:
            cislo = '' # vyprázdnit řetězec pro střeádání znaků čísla
            if znak.isspace():
                stav = PRED_CISLEM; znak = file.read(1); continue
            elif znak == '+' or znak == '-':
                cislo += znak
                stav = ZA_ZNAMENKEM; znak = file.read(1); continue
            elif znak == '.':
                cislo += znak
                stav = ZLOMEK; znak = file.read(1); continue
            elif znak.isdigit():
                cislo += znak
                stav = CELKY; znak = file.read(1); continue
            else:
                print(f"\n      chybný znak v {pocet+1}. čísle "
                      f"ve stavu PRED_CISLEM: {repr(znak)}")
```

```

        stav = PRED_CISLEM; znak = file.read(1); continue
elif stav == ZA_ZNAMENKEM:
    if znak.isdigit(): # číslice
        cislo += znak
        stav = CELKY; znak = file.read(1); continue
    elif znak == '.': # tečka
        cislo += znak
        stav = ZLOMEK; znak = file.read(1); continue
    else:
        print(f"\n    chybný znak v {pocet+1}. čísle "
              f"ve stavu ZA_ZNAMENKEM: {repr(znak)}")
        stav = PRED_CISLEM; continue # nečtu další znak
elif stav == CELKY:
    if znak.isdigit():
        cislo += znak
        stav = CELKY; znak = file.read(1); continue
    elif znak == ".":
        cislo += znak
        stav = ZLOMEK; znak = file.read(1); continue
    elif znak == ',':
        zpracovani() # výpočet souhrnů
        stav = PRED_CISLEM; znak = file.read(1); continue
    elif znak.isspace():
        stav = ZA_CISLEM; znak = file.read(1); continue
    else:
        print(f"\n    chybný znak v {pocet+1}. čísle "
              f"ve stavu CELKY: {repr(znak)}")
        stav = CELKY; znak = file.read(1); continue
elif stav == ZLOMEK:
    if znak.isdigit():
        cislo += znak
        stav = ZLOMEK; znak = file.read(1); continue
    elif znak == ',':
        zpracovani() # výpočet souhrnů
        stav = PRED_CISLEM; znak = file.read(1); continue
    elif znak.isspace():
        stav = ZA_CISLEM; znak = file.read(1); continue
    else:
        print(f"\n    chybný znak v {pocet+1}. čísle "
              f"ve stavu ZLOMEK: {repr(znak)}")
        stav = ZLOMEK; znak = file.read(1); continue
elif stav == ZA_CISLEM:
    if znak.isspace():
        stav = ZA_CISLEM; znak = file.read(1); continue
    elif znak == ',':
        zpracovani() # výpočet souhrnů
        stav = PRED_CISLEM; znak = file.read(1); continue
    else:
        print(f"\n    chybný znak v {pocet+1}. čísle "
              f"ve stavu ZA_CISLEM: {repr(znak)}")
        stav = PRED_CISLEM; znak = file.read(1); continue

# Konec hlavního cyklu:
if stav != PRED_CISLEM: # když není stav před číslem (chybí čárka)
    zpracovani() # ještě jednou vypočtu souhrny
print('\nOčištěná znaková čísla:', scisla)
if pocet == 0:
    pocet = 1
return {"maximum": maximum, "minimum": minimum, "průměr": soucet/pocet}

```

Vysvětlivky k funkci

Funkce přijímá vstupní data z textového souboru a čte znaky po jednom metodou `read(1)`.

V pomocné funkci `zpracovani()` převede řetězec na *číslo* a počítá maximum, minimum, součet a počet čísel. Jestliže řetězec obsahuje aspoň jednu číslici, zapíše číslo do seznamu. Jinak hlásí chybu.

Příklad bez chyb

```
file = open("cisla.txt", "w")
file.write("-23.5, 42.777 , -84, .72, 42 ")
file.close()
file = open("cisla.txt", "r")

vysledky = cisla_bez_exponentu(file)

Čtené znaky: -23.5, 42.777 , -84, .72, 42
Očištěná znaková čísla: ['-23.5', '42.777', '-84', '.72', '42']

print(vysledky)
{'maximum': 42.777, 'minimum': -84.0, 'průměr': -4.4006}
```

Příklad s chybami

```
# zápis zkušebních dat do souboru
file = open("cisla.txt", "w")
file.write("- 23.5, +42.7t77 , -. 84, .72., 42c ")
file.close()

# otevření souboru čísel
file = open("cisla.txt", "r")

vysledky = cisla_bez_exponentu(file)

Čtené znaky: -
chybný znak v 1. čísle ve stavu ZA_ZNAMENKEM: ' '
23.5, +42.7t
chybný znak v 2. čísle ve stavu ZLOMEK: 't'
77 , -. 8
chybný znak v 3. čísle ve stavu ZA_CISLEM: '8'
4
chybný znak v 3. čísle ve stavu ZA_CISLEM: '4'
'
neúplné 3. číslo: '-.'
.72.
chybný znak v 3. čísle ve stavu ZLOMEK: '.'
, 42c
chybný znak v 4. čísle ve stavu CELKY: 'c'

Očištěná znaková čísla: ['23.5', '+42.777', '.72', '42']

print(vysledky)
{'maximum': 42.777, 'minimum': 0.72, 'průměr': 27.24925}
```

Kontrola zlomkových čísel s exponentem

Konečně bude naším úkolem opět hledat v *textovém souboru* nebo v prostém *textovém řetězci* podřetězce oddělené *právě jednou čárkou* a možnými mezerami (bílémi znaky – whitespace). Zároveň budeme kontrolovat nalezené podřetězce zbavené oddělovačů tak, aby tvořily celé číslo nebo zlomkové číslo typu `float` podle formální definice. Nyní však budeme počítat také s tzv. vědeckým tvarem čísla s *exponentem* (např. $1.234e5$, tedy 1.234×10^5).

Chybné znaky ohlásíme tiskem a po jejich vynechání se pokusíme získat platné číslo a zpracovat je pro výpočet maxima, minima, součtu a průměru. Tyto hodnoty budeme tisknout průběžně po každém zkontrolovaném čísle a také na konci.

Formální definice zlomkového čísla je vyjádřena následujícími syntaktickými pravidly.

```
cislo_se_znamenkem ::= ['+' | '-'] cislo
cislo                ::= celky | zlomkove_cislo | exponentove_cislo
zlomkove_cislo       ::= [celky] zlomek | celky '.'
exponentove_cislo    ::= (celky | zlomkove_cislo) exponent
celky                ::= cislice+
zlomek               ::= '.' celky
exponent             ::= ('e' | 'E') ['+' | '-'] celky
číslice              ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
```

Znaménko `+` za výrazem (*číslice*`+`) znamená, že výraz musí být přítomen aspoň jednou a může se libovolně opakovat. Hvězdička `*` za výrazem znamená, že výraz se může libovolně opakovat, ale nemusí být vůbec přítomen. Svislá čárka `|` znamená "nebo". Výraz v hranatých závorkách `[ ]` je nepovinný. Výraz v kulatých závorkách `( )` je povinná skupina. Některé značky se shodují se značkami používanými v regulárních výrazech (plus, hvězdička, svislá čárka, kulaté závorky) a některé se používají jinak (zdejší hranaté závorky jsou v regulárním výrazu vyjádřeny otazníkem).

Regulární výraz této definice může vypadat v rozepsaném (verbose) tvaru takto:

```
r'''[+-]?          # nepovinné znaménko
(                 # exponentové číslo
(                 # celky nebo zlomkové číslo
(                 # zlomkové číslo
(\d*[.]\d+)      # nepovinné celky se zlomkem
|
\d+[.]           # celky s tečkou
)                 # konec zlomkového čísla
|
\d+              # celky
)                 # konec celků nebo zlomkového čísla
(                 # exponent
[eE]             # písmeno e nebo E
[+-]?            # nepovinné znaménko
\d+              # celky
)                 # konec exponentu
)                 # konec exponentového čísla
|
(                 # zlomkové číslo
(\d+)?[.]\d+     # nepovinné celky se zlomkem
|
\d+[.]           # celky s tečkou
)                 # konec zlomkového čísla
'''
```


Neformálně, číslo musí obsahovat aspoň jednu číslici, může mít tečku uvnitř, na začátku nebo na konci (nebo nikde), nulu na začátku nebo na konci, např. 01.10 .10 01. 01
Dále může mít na konci exponent, např. 3.45e+3 2E-3 1e0

Čísla oddělená čárkou a mezerami

Takto definovaná čísla budeme hledat v textu, kde *mezi čísly je jedna povinná čárka* a libovolný (i nulový) počet mezer. Na konci textu nemusí být za číslem čárka. Text může být také prázdný. To je formálně vyjádřeno těmito pravidly:

```
whitespace ::= ' ' | '\n' | '\t' | '\r' | '\v' | '\f'
mezery ::= whitespace*
text ::= ([mezery] číslo [mezery] ',')* [[mezery] číslo [mezery]]
```

Regulární výraz této definice, v němž symbol *číslo* představuje dříve uvedený regulární výraz, vypadá takto:

```
r'(\s*(číslo)\s*[,])*(\s*(číslo)\s*)?'
```

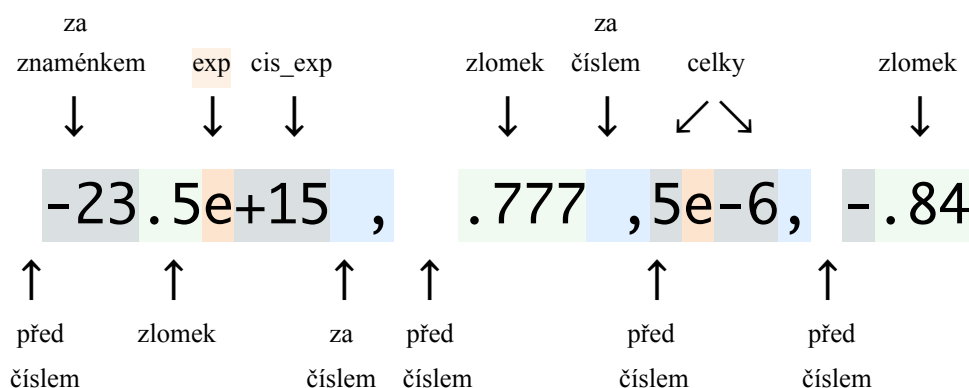
Hrubý popis činnosti stavového stroje

Zpracování proběhne v cyklu tak, že z řetězce nebo souboru se čte znak po znaku. Program přitom bude rozeznávat následující stavy:

PRED_CISLEM	na začátku a za čárkou
CELKY	bylo přečteno znaménko nebo číslice
ZLOMEK	byla přečtena tečka
EXP	bylo přečteno písmeno e nebo E
CIS_EXP	bylo přečteno znaménko nebo číslice ve stavu EXP
ZA_CISLEM	byla přečtena mezera nebo čárka
konec vstupu	

V každém stavu program vyhodnotí a zpracuje přečtený znak, načež přečte další znak (nebo někdy nepřečte) a podle něj přejde do stejného nebo jiného stavu. Při vyhodnocování povolený znak zařadí do čísla, nepovolený znak ohlásí jako chybu, mezera (whitespace), čárka a nepovolený znak vynechá. Vždy, když se přečte čárka, shromážděné povolené znaky se zpracují jako číslo. Číslo se také zpracuje, i když za ním na konci textu nebyla přečtena čárka.

Příklad textu



Podrobný popis stavového stroje

Začíná stavem PRED_CISLEM a čte první znak.

Když byl ve stavu

PRED_CISLEM přečten znak

mezera, přečte další znak a přejde do stavu PRED_CISLEM
+ nebo **-**, zařadí znak do čísla, přečte další znak a přejde do stavu CELKY
tečka, zařadí znak do čísla, přečte další znak a přejde do stavu ZLOMEK
číslice, zařadí znak do čísla a přejde do stavu CELKY (*nečte další znak*)
jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu PRED_CISLEM

CELKY přečten znak

číslice, zařadí znak do čísla, přečte další znak a zůstane ve stavu CELKY
tečka, zařadí znak do čísla, přečte další znak a přejde do stavu ZLOMEK
písmeno e nebo E, zařadí znak do čísla, přečte další znak a přejde do stavu EXP
mezera, přečte další znak a přejde do stavu ZA_CISLEM
čárka, *zpracuje* číslo, přečte další znak a přejde do stavu PRED_CISLEM
jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu CELKY

ZLOMEK přečten znak

číslice, zařadí znak do čísla, přečte další znak a zůstane ve stavu ZLOMEK
písmeno e nebo E, zařadí znak do čísla, přečte další znak a přejde do stavu EXP
mezera, přečte další znak a přejde do stavu ZA_CISLEM
čárka, *zpracuje* číslo, přečte další znak a přejde do stavu PRED_CISLEM
jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu ZLOMEK

EXP přečten znak

+ nebo **-**, zařadí znak do čísla, přečte další znak a přejde do stavu CIS_EXP
číslice, zařadí znak do čísla a přejde do stavu CIS_EXP (*nečte další znak*)
jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu EXP

CIS_EXP přečten znak

číslice, zařadí znak do čísla, přečte další znak a zůstane ve stavu CIS_EXP
mezera, přečte další znak a přejde do stavu ZA_CISLEM
čárka, *zpracuje* číslo, přečte další znak a přejde do stavu PRED_CISLEM
jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu CIS_EXP

ZA_CISLEM přečten znak

mezera, přečte další znak a přejde do stavu ZA_CISLEM
čárka, *zpracuje* číslo, přečte další znak a přejde do stavu PRED_CISLEM
jiný znak, ohlásí chybu, přečte další znak a zůstane ve stavu ZA_CISLEM

Když cyklus skončí v jiném stavu než PRED_CISLEM, tedy, když chyběla poslední čárka, *zpracuje* se ještě poslední číslo.

Zpracování spočívá v dokončení čísla sestaveného z povolených čtených znaků:

- kontrola *úplnosti* čísla (zda nechybí číslice),
- zařazení čísla do seznamu,
- převod do čísla typu `float`,
- výpočet průběžného maxima, minima, součtu a počtu čísel.

Číslo s podtržítkem

Mezi povolené znaky v čísle se od verze 3.6 Pythonu řadí *podtržítka*, ale nejvýše jedno mezi dvěma číslicemi. Definici celků

```
celky ::= cislice+  
\\d+
```

nahradíme novou:

```
celky ::= číslice (['_'] číslice)*  
\\d(_?\\d)*
```

Tento požadavek komplikuje kontrolu čísla, protože k ní nestačí zkoumat jen právě přečtený znak podtržítka, ale také minulý znak a budoucí znak (přečtený dopředu), tedy *kontext*. K řešení takové kontroly existují v podstatě dvě metody.

- *Zavedení logické proměnné – přepínače* (byla přečtena číslice / nebyla přečtena číslice).
 1. Na začátku stavu PŘED_ČÍSLEM se přepínač *vypne*.
 2. Vždy po přečtení číslice se přepínač *zapne*.
 3. Přečtené podtržítka se uloží do pomocné proměnné a *přečte se následující znak*.
 - Je-li ten znak číslice a přepínač je *zapnutý*, uložené podtržítka do sestavovaného čísla *nezařazujeme*, jinak se hlásí chyba. V obou případech se přepínač *vypne*.
 - Výpočet zůstává ve stejném stavu, ale *nečte se další znak*, protože ten už byl přečten.
 4. Po přečtení jiného znaku se přepínač *vypne*.
- *Čtení dopředu* (read ahead). Místo abychom na začátku četli jen jeden znak, čteme tři znaky. Příště čteme další znak a předchozí dva si pamatujeme. To umožňuje po každém čtení dalšího znaku porovnávat tři znaky.
 1. Je-li první znak číslice, druhý podtržítka a třetí číslice, je text správný, ale podtržítka do sestavovaného čísla *nezařazujeme*. Není-li třetí znak číslice, hlásí se na podtržítka chyba.
 2. Přečte se *dvakrát další znak*, čímž se přeskočí podtržítka.

V následující funkci bude stavový stroj takto *modifikován* ve srovnání s předchozím podrobným popisem. K vyřešení podtržitek použijeme čtení tří znaků dopředu a pozměníme stavy CELKY, ZLOMEK a CIS_EXP. Výsledné číslo zbavíme podtržitek.

Výsledný program je teď pochopitelně mnohem delší než předchozí bez exponentu a podtržítka.

Funkce se stavovým strojem

```
def ciska_s_exponentem(
    indata, # vstup dat: string nebo otevřený vstupní soubor
    outdata = None, # otevřený výstupní soubor nebo None
    sep: str = ',', # oddělovací znak může být třeba: ',', ';', '|', '\t', '\n', 'a', '😊' | ...
    linesep: str = '\n', # oddělovací znak řádků: '\n', ';', | ...
    dec: bool = False, # převod do 'float' ( True - do desítkových čísel)
):
    """*** Funkce zjišťuje čísla ve znakovém tvaru a počítá z nich
    maximum, minimum, součet a průměr. """

    # Parametr indata je buď otevřený vstupní soubor nebo string.
    # Parametr outdata je buď otevřený vstupní soubor nebo None.
    # Parametr sep je jednoznakový oddělovač čísel: čárka, středník, mezera apod.
    # Parametr linesep je jednoznakový oddělovač řádků: čárka, středník, mezera apod.
    # Parametr dec=False určuje, že výsledná čísla budou typu 'float'
    # dec=True určuje, že výsledná čísla budou typu 'Decimal'
    assert hasattr(indata, "read") or type(indata) == str, \
        "Vstup musí být buď text nebo otevřený vstupní soubor."
    assert hasattr(outdata, "write") or outdata is None, \
        "Výstup musí být buď otevřený výstupní soubor nebo None."
    assert type(sep) == str and len(sep) == 1, \
        "Oddělovač čísel musí být jeden znak."
    assert type(linesep) == str and len(linesep) == 1 \
        or linesep is None, \
        "Oddělovač řádků musí být jeden nebo dva znaky."
    assert dec or not dec or dec is None, \
        "Parametr 'desítková čísla' musí být True, False nebo None."

    seznam_cisel = [] # seznam očištěných čísel ve znakovém tvaru
    if dec:
        maximum = decimal.Decimal('-inf')
        minimum = decimal.Decimal('inf')
        soucet = decimal.Decimal('0.0') # součet zpracovaných čísel
        pocet = decimal.Decimal('0') # počet zpracovaných čísel
    else:
        maximum = float('-inf')
        minimum = float('inf')
        soucet = 0.0 # součet zpracovaných čísel
        pocet = 0 # počet zpracovaných čísel
    cislo = '' # řetězec pro strádání znaků čísla
    tecka = '.' # desetinná tečka
    PRED_CISLEM = 1
    ZA_ZNAMENKEM = 2
    CELKY = 3
    ZLOMEK = 4
    EXP = 5
    CIS_EXP = 6
    ZA_CISLEM = 7
    znak, znak2, znak3 = ',', '.', ' ' # znaky čtené dopředu
    ind = 0 # index znaku v textu

    # zpracování čísla
    def zpracovani(lsep=None):
        nonlocal cislo, maximum, minimum, soucet, pocet, znak2
        if any(char.isdigit() for char in cislo): # řetězec obsahuje aspoň jednu číslici
            seznam_cisel.append(cislo)
            if dec:
                fl = decimal.Decimal(cislo)
            else:
                fl = float(cislo)
            if fl > maximum:
                maximum = fl
            if fl < minimum:
                minimum = fl
            soucet += fl
            if hasattr(outdata, "write"): # zápis do souboru
                if znak2 == ' ': # poslední číslo? (druhý znak je konec dat)
                    outdata.write(f"{fl:n}") # za číslem už nebudou oddělovače
```

```

        elif lsep is None:
            outdata.write(f"{fl:n}{sep}" ) # + oddělovač čísel
        else: # + oddělovač čísel + oddělovač řádků
            outdata.write(f"{fl:n}{sep}{lsep}")
        pocet += 1
    else:
        chyba(f"chybné číslo: řetězec {repr(cislo)} "
              f"neobsahuje žádnou číslici; vynechá se")
    print(cislo, end='')
    print(f"{' ' * (15 - len(cislo))}{pocet}{sep} {maximum:n}{sep} "
          f"{minimum:n}{sep} {soucet:n}{sep} {soucet/pocet:n}")
    cislo = '' # vyprázdním řetězec pro strádání znaků dalšího čísla

def jemezera(character):
    if character.isspace() and character is not linesep:
        return True
    else:
        return False

def chyba(text_chyby):
    print(text_chyby)

# funkce přečte další tři znaky o 1 dopředu
def dalsi():
    nonlocal indata, znak, znak2, znak3, ind
    if hasattr(indata, "read"): # vstup je otevřený vstupní soubor?
        znak = znak2 # znak znak byl už přečten
        znak2 = znak3
        znak3 = indata.read(1)
    else: # vstup je text?
        if ind < len(indata):
            znak = indata[ind]
        else:
            znak = '' # příznak konce dat pro hlavní cyklus
        if ind + 1 < len(indata):
            znak2 = indata[ind + 1]
        else:
            znak2 = '' # příznak konce dat pro hlavní cyklus
        if ind + 2 < len(indata):
            znak3 = indata[ind + 2]
        else:
            znak3 = '' # příznak konce dat pro hlavní cyklus
        ind += 1

# Tělo hlavní funkce

# na začátku čtu první tři znaky
if hasattr(indata, "read"): # vstup je otevřený vstupní soubor?
    # vstup je soubor
    znak = indata.read(1) # čtu první znak
    if znak == '': # je to prázdný znak - "konec souboru"?
        return f"počet: {pocet:n}{sep} maximum: {maximum:n}{sep} " \
               f"minimum: {minimum:n}{sep} součet: {soucet:n}{sep} průměr: {None}"
    # před zahájením cyklu přečtu první tři znaky, pokud existují
    znak2 = indata.read(1)
    znak3 = indata.read(1)
else: # vstup je string
    if len(indata) == 0:
        return f"počet: {pocet:n}{sep} maximum: {maximum:n}{sep} " \
               f"minimum: {minimum:n}{sep} součet: {soucet:n}{sep} průměr: {None}"
    ind = 0
    dalsi() # před zahájením cyklu čtu další znak

# vyprázdním řetězec pro strádání znaků čísla
cislo = ''

stav = PRED_CISLEM # nastavím počáteční stav
print("Opravená čísla: ")

# cyklus stavového stroje
while True:
    if znak == '': # prázdný znak značí konec souboru
        break # už ho nezpracuji, aby se na něm nehlásila chyba, a končím

```

```

if stav == PRED_CISLEM:
    if jemezera(znak): # mezera?
        stav = PRED_CISLEM
        dalsi()
        continue
    elif znak == '+' or znak == '-': # znaménko?
        cislo += znak
        stav = ZA_ZNAMENKEM
        dalsi()
        continue
    elif znak.isdigit(): # číslice?
        cislo += znak
        stav = CELKY
        dalsi()
        continue
    elif znak == '.':
        cislo += znak
        stav = ZLOMEK
        dalsi()
        continue
    elif znak == linesep: # oddělovač řádků? přeskočíme ho
        dalsi()
        continue
    else: # chyba
        chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)} "
              f"ve stavu PRED_CISLEM 3 v {pocet + 1}. čísle")
        dalsi()
        continue
elif stav == ZA_ZNAMENKEM:
    if znak.isdigit(): # číslice
        cislo += znak
        stav = CELKY
        dalsi()
        continue
    elif znak == '.': # tečka
        cislo += znak
        stav = ZLOMEK
        dalsi()
        continue
    else:
        chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)} "
              f"ve stavu ZA_ZNAMENKEM v {pocet + 1}. čísle")
        stav = PRED_CISLEM
        dalsi()
        continue
elif stav == CELKY:
    if znak.isdigit(): # číslice?
        cislo += znak
        if znak2 == '_': # druhý znak je podtržítko?
            if znak3.isdigit(): # a třetí znak je číslice?
                pass # podtržítko znak2 NEpřijde do čísla
            else:
                chyba(f"          chybný znak {repr(znak2)} "
                      f"za znakem {repr(znak)} "
                      f"ve stavu CELKY 1 v {pocet + 1}. čísle")
                # přeskočím znak podtržítko (čili další znak přečtu dvakrát)
                dalsi()
                dalsi()
                continue
            dalsi()
            continue
    elif znak == '_' and not znak2.isdigit():
        # první je podtržítko a druhý není číslice?
        chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)} "
              f"ve stavu CELKY 2 v {pocet + 1}. čísle")
        dalsi()
        continue
    elif znak == '_' and znak2.isdigit():
        # první je podtržítko a druhý je číslice?
        dalsi() # přeskočím ho
        continue

```

```

elif znak == tečka: # tečka?
    cislo += znak
    stav = ZLOMEK
    dalsi()
    continue
elif znak == 'e' or znak == 'E': # exponent?
    cislo += znak
    stav = EXP
    dalsi()
    continue
elif znak == sep: # oddělovač čísel?
    zpracovani()
    stav = PRED_CISLEM
    dalsi()
    continue
elif znak == linesep: # oddělovač řádků?
    zpracovani(linesep) # zpracování s výstupem oddělovače řádků
    stav = PRED_CISLEM
    dalsi()
    continue
elif jemezera(znak):
    stav = ZA_CISLEM
    dalsi()
    continue
else:
    chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)} "
          f"ve stavu CELKY v {pocet + 1}. čísle")
    dalsi()
    continue
elif stav == ZLOMEK:
    if znak.isdigit(): # číslice?
        cislo += znak
        if znak2 == '_': # druhý znak je podtržítko?
            if znak3.isdigit(): # a třetí znak je číslice?
                pass # podtržítko znak2 NEpřijde do čísla
            else:
                chyba(f"          chybný znak {repr(znak2)} "
                      f"za znakem {repr(znak)} "
                      f"ve stavu ZLOMEK 1 v {pocet + 1}. čísle")
                # přeskočím znak podtržítko (další znaky přečtu dvakrát)
                dalsi()
                dalsi()
                continue
        dalsi()
        continue
    elif znak == '_' and not znak2.isdigit(): # podtržítko?
        # první je podtržítko a druhý není číslice?
        chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)} "
              f"ve stavu ZLOMEK 2 v {pocet + 1}. čísle")
        dalsi()
        continue
    elif znak == 'e' or znak == 'E': # exponent?
        cislo += znak
        stav = EXP
        dalsi()
        continue
    elif znak == sep: # oddělovač čísel?
        zpracovani()
        stav = PRED_CISLEM
        dalsi()
        continue
    elif jemezera(znak): # mezera?
        stav = ZA_CISLEM
        dalsi()
        continue
    elif znak == linesep: # oddělovač řádků?
        zpracovani(linesep) # zpracování s výstupem oddělovače řádků
        stav = PRED_CISLEM
        dalsi()
        continue
    else: # chyba
        chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)} "
              f"ve stavu ZLOMEK v {pocet + 1}. čísle")

```

```

stav = ZLOMEK
dalsi()
continue

elif stav == EXP:
    if znak == '+' or znak == '-':
        cislo += znak
        stav = CIS_EXP
        dalsi()
        continue
    elif znak.isdigit():
        stav = CIS_EXP
        continue # nečtu další
    else:
        chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)} "
              f"ve stavu EXP v {pocet + 1}. čísle")
        dalsi()
        continue
elif stav == CIS_EXP:
    if znak.isdigit(): # číslice?
        cislo += znak
        if znak2 == '_': # druhý znak je podtržítko?
            if znak3.isdigit(): # a třetí znak je číslice?
                pass # podtržítko znak2 NEpřijde do čísla
            else:
                chyba(f"          chybný znak {repr(znak2)} "
                      f"za znakem {repr(znak)}"
                      f" ve stavu CIS_EXP 1 v {pocet + 1}. čísle")
                # přeskočím znak podtržítko (další znaky přečtu dvakrát)
                dalsi()
                dalsi()
                continue
        dalsi()
        continue
    elif znak == '_' and not znak2.isdigit(): # podtržítko?
        # první je podtržítko a druhý není číslice?
        chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)}"
              f" ve stavu CIS_EXP 2 v {pocet + 1}. čísle")
        dalsi()
        continue
    elif znak == '_' and znak2.isdigit():
        # první je podtržítko a druhý je číslice?
        dalsi() # přeskočím ho
        continue
    elif znak == sep: # oddělovač čísel?
        zpracovani()
        stav = PRED_CISLEM
        dalsi()
        continue
    elif jemezera(znak): # mezera?
        stav = ZA_CISLEM
        dalsi()
        continue
    elif znak == linesep: # oddělovač řádků?
        zpracovani(linesep) # zpracování s výstupem oddělovače řádků
        stav = PRED_CISLEM
        dalsi()
        continue
    else: # chyba
        chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)}"
              f" ve stavu CIS_EXP v {pocet + 1}. čísle")
        dalsi()
        continue
elif stav == ZA_CISLEM:
    if jemezera(znak): # mezera?
        dalsi()
        continue
    elif znak == sep: # oddělovač čísel?
        zpracovani()
        stav = PRED_CISLEM
        dalsi()
        continue

```



```

elif znak == linesep: # oddělovač řádků?
    zpracovani(linesep) # zpracování s výstupem oddělovače řádků
    stav = PRED_CISLEM
    dalsi()
    continue
else: # chyba
    chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)}"
          f" ve stavu ZA_CISLEM v {pocet + 1}. čísle")
    dalsi()
    continue

# konec cyklu:
if stav != PRED_CISLEM: # když teď není stav před číslem (chybí čárka)
    zpracovani() # zpracuji poslední číslo
print('\nOčištěná znaková čísla:', seznam_cisel)
if pocet == 0:
    pocet = 1
return f"počet: {pocet:n}{sep} maximum: {maximum:n}{sep} minimum: " \
       f"{minimum:n}{sep} součet: {soucet:n}{sep} průměr: {soucet/pocet:n}"

```

Vysvětlivky k funkci

Funkce přijímá několik parametrů, které umožňují

- vstup z otevřeného vstupního souboru nebo z textu (indata),
- výstup očištěných dat do otevřeného výstupního souboru (outdata)
- volbu oddělovače čísel místo čárky (sep)
- volbu oddělovače řádků (linesep)
- volbu desítkové aritmetiky pro převod řetězců do čísel a výpočty (dec)

Kontrola podtržítok

Funkce využívá tří znaků znak1, znak2, znak3 čtených dopředu ke kontrole, zda podtržítka je mezi dvěma číslicemi. Když ano, nehlásí chybu, ale podtržítka nepřijme do čísla; další znak pak přečte dvakrát. Když podtržítka není mezi dvěma číslicemi, hlásí chybu. Téměř totožný úsek programu je použit ve stavech CELKY, ZLOMEK a CIS_EXP, aby se případná chyba hlásila i s údajem o stavu a pořadí kontroly.

```

...
elif stav == CELKY:
    if znak.isdigit(): # číslice?
        cislo += znak
        if znak2 == ' ': # druhý znak je podtržítka?
            if znak3.isdigit(): # a třetí znak je číslice?
                pass # podtržítka znak2 NEpřijde do čísla
            else:
                chyba(f"          chybný znak {repr(znak2)} "
                      f"za znakem {repr(znak)} "
                      f"ve stavu CELKY 1 v {pocet + 1}. čísle")
                # přeskočím znak podtržítka (čili další znak přečtu dvakrát)
                dalsi()
                dalsi()
                continue
        dalsi()
        continue
    elif znak == '_' and not znak2.isdigit():
        # první je podtržítka a druhý není číslice?
        chyba(f"          chybný znak {repr(znak)} před {repr(znak2 + znak3)} "
              f"ve stavu CELKY 2 v {pocet + 1}. čísle")
        dalsi()
        continue
    elif znak == '.' and znak2.isdigit():
        # první je podtržítka a druhý je číslice?
        dalsi() # přeskočím ho
        continue
...

```

Čtení dalšího znaku

Čtení dalšího znaku se řídí podle parametru `indata` pro vstup dat. Otevřený vstupní *soubor* funkce pozná podle toho, že parametr obsahuje atribut `"read"`. Když ne, funkce volí *text*; délka textu je známá a může tedy číst další znak podle indexu.

```
def dalsi():
    nonlocal indata, znak1, znak2, znak3, ind
    if hasattr(indata, "read"): # vstup je otevřený vstupní soubor?
        znak1 = znak2 # znak znak1 byl už přečten
        znak2 = znak3
        znak3 = indata.read(1)
    else: # vstup je text?
        if ind < len(indata):
            znak1 = indata[ind]
        else:
            znak1 = '' # příznak konce dat pro hlavní cyklus
            if ind + 1 < len(indata):
                znak2 = indata[ind + 1]
            else:
                znak2 = '' # příznak konce dat pro hlavní cyklus
            if ind + 2 < len(indata):
                znak3 = indata[ind + 2]
            else:
                znak3 = '' # příznak konce dat pro hlavní cyklus
        ind += 1
```

Čtení znaků na začátku

Na začátku před spuštěním hlavního cyklu stavového stroje se přečtou první tři znaky ze souboru metodou `read()`, je-li vstupem soubor. Z textu se přečtou další tři znaky pomocí funkce `dalsi()`, je-li vstupem text. Případně se ohlásí prázdný vstup.

```
...
if hasattr(indata, "read"): # vstup je otevřený vstupní soubor?
    # vstup je soubor
    znak1 = indata.read(1) # čtu první znak
    if znak1 == '': # je to prázdný znak - "konec souboru"?
        return f"počet: {pocet:n}{sep} maximum: {maximum:n}{sep} " \
               f"minimum: {minimum:n}{sep} součet: {soucet:n}{sep} průměr: {None}"
    znak2 = indata.read(1) # čtu druhý znak
    znak3 = indata.read(1) # čtu třetí znak
else: # vstup je string
    if len(indata) == 0:
        return f"počet: {pocet:n}{sep} maximum: {maximum:n}{sep} " \
               f"minimum: {minimum:n}{sep} součet: {soucet:n}{sep} průměr: {None}"
    ind = 0
    dalsi() # čtu další znak
...
```

Prázdný vstup se ohlásí vrácením zprávy

```
počet: 0; maximum: -inf; minimum: inf; součet: 0; průměr: None
```

Zpracování čísel

Ve funkci `zpracovani()` se počítá maximum, minimum a součet buď jako typ `float` nebo `Decimal`. Jestliže řetězec obsahuje aspoň jednu číslici, zapíše číslo do seznamu. Mají-li čísla vystoupit do souboru, bere se v úvahu formátování podle lokalizace.

Lokalizace

Před voláním funkce lze zvolit např. českou lokalizaci použitím modulu `locale`. Ta má smysl jen když oddělovačem čísel nebude čárka, protože ta odděluje desetinná místa. Pak je nejlepší volit středník za oddělovač čísel v parametru `sep`.

```
import locale
locale.setlocale(locale.LC_ALL, 'cs_CZ')
```

Ve funkci jsou použity příkazy pro výstup formátovaného čísla do souboru, kde typ **n** za dvojtečkou znamená lokalizovanou úpravu.

```
...
if hasattr(outdata, "write"): # zápis do souboru
    if znak2 == ':': # poslední číslo? (druhý znak je konec dat)
        outdata.write(f"{f1:n}") # za číslem už nebudou oddělovače
    elif lsep is None:
        outdata.write(f"{f1:n}{sep}" ) # + oddělovač čísel
    else: # + oddělovač čísel + oddělovač řádků
        outdata.write(f"{f1:n}{sep}{lsep}")
...
```

Zkouška chyb

Příprava vstupních dat

```
text = f"-23.5, + 42.7t77 , -. . 84, .b72., 42c , ."
```

Spuštění funkce

```
outfile = open("cisla_text.csv", "w+") # otevřu prázdný výstupní soubor
vysledky = cisla_s_exponentem(indata=text, outdata=outfile, sep=',')
```

Tisk průběžných výsledků

```
Opravená čísla:
-23.5          1, -23.5, -23.5, -23.5, -23.5
chybný znak ' ' před '42' ve stavu ZA_ZNAMENKEM v 2. čísle
chybný znak 't' před '77' ve stavu ZLOMEK v 2. čísle
+42.777        2, 42.777, -23.5, 19.277, 9.6385
chybný znak '.' před '8' ve stavu ZA_CISLEM v 3. čísle
chybný znak '8' před '4,' ve stavu ZA_CISLEM v 3. čísle
chybný znak '4' před ', ' ve stavu ZA_CISLEM v 3. čísle
chybné číslo: řetězec '-.' neobsahuje žádnou číslici; vynechá se
-.             2, 42.777, -23.5, 19.277, 9.6385
chybný znak 'b' před '72' ve stavu ZLOMEK v 3. čísle
chybný znak '.' před ', ' ve stavu ZLOMEK v 3. čísle
.72            3, 42.777, -23.5, 19.997, 6.66567
chybný znak 'c' před ', ' ve stavu CELKY v 4. čísle
42             4, 42.777, -23.5, 61.997, 15.4992
chybné číslo: řetězec '.' neobsahuje žádnou číslici; vynechá se
.             4, 42.777, -23.5, 61.997, 15.4992
```

```
Očištěná znaková čísla: ['-23.5', '+42.777', '.72', '42']
```

Tisk vrácených výsledků

```
print(vysledky)
počet: 4, maximum: 42.777, minimum: -23.5, součet: 61.997, průměr: 15.4992
```

Výstupní soubor

```
-23.5,42.777,-0.84,0.72,42,
```

Příklad s desítkovými výsledky

Příprava vstupních dat

```
import decimal

# zápis zkušebních dat do souboru
# -----
with open("cisla.txt", "w") as outfile:
    outfile.write(f"-23.5e3, -.32, 42.777\n2_0_0_0, *19, 6 ,42\n")

# vstup je otevřený soubor
# -----
infile = open("cisla.txt", "r")
print(repr(f"Soubor:{infile.read()}")) # otisknu soubor čtený jako celek

'Soubor:-23.5e3, -.32, 42.777 \n2_0_0_0, *19, 6 ,42\n'

infile.seek(0) # vrátím začátek souboru

# výstup čísel do souboru s oddělovačem řádků
outfile = open("cisla_file.csv", "w+") # otevřu prázdný výstupní soubor
```

Spuštění funkce pro desítkové výsledky

```
vysledky = cisla_s_exponentem(indata=infile, outdata=outfile, sep=',',
                             linesep='\n', dec=True)
```

Tisk průběžných výsledků

```
Opravená čísla:
-23.5e3      1, -2.35e+4, -2.35e+4, -23500.0, -23500.0
-.32         2, -0.32, -2.35e+4, -23500.32, -11750.16
42.777       3, 42.777, -2.35e+4, -23457.543, -7819.181
2000         4, 2000, -2.35e+4, -21457.543, -5364.38575
             chybný znak '*' před '19' ve stavu PRED_CISLEM 3 v 5. čísle
19           5, 2000, -2.35e+4, -21438.543, -4287.7086
6            6, 2000, -2.35e+4, -21432.543, -3572.0905
42           7, 2000, -2.35e+4, -21390.543, -3055.791857142857142857142857

Očištěná znaková čísla: ['-23.5e3', '-.32', '42.777', '2000', '19', '6', '42']
```

Tisk vrácených výsledků

```
print(vysledky)
počet: 7, maximum: 2000, minimum: -2.35e+4, součet: -21390.543, průměr:
-3055.791857142857142857142857

outfile.close() # uzavřu výstupní soubor
infile.close() # nepovinně uzavřu soubor
```

Výstupní soubor cisla_file.csv

```
-2.35e+4,-0.32,42.777,
2000,19,6,42
```

Příklad s českou lokalizací

Příprava vstupních dat

```
import locale
locale.setlocale(locale.LC_ALL, 'cs_CZ') # jen se středníkem jako oddělovačem čísel

# vstupem je zkušební text se středníky
# -----
text = f"""
-23.5e3; -.32; 42.777
2_0_0_0; *19; 6 ;42
"""

print("Soubor:", repr(text))
Soubor: '\n-23.5e3; -.32; 42.777 \n2_0_0_0; *19; 6 ;42\n'

outfile = open("cisla_text.csv", "w+") # otevřu prázdný výstupní soubor
```

Spuštění funkce

```
vysledky = cisla_s_exponentem(indata=text, outdata=outfile, sep=';')
```

Tisk průběžných výsledků s desetinnými čárkami

```
Opravená čísla:
-23.5e3      1; -23 500; -23 500; -23 500; -23 500
-.32         2; -0,32; -23 500; -23 500,3; -11 750,2
42.777       3; 42,777; -23 500; -23 457,5; -7 819,18
2000         4; 2 000; -23 500; -21 457,5; -5 364,39
             chybný znak '*' před '19' ve stavu PRED_CISLEM 3 v 5. čísle
19           5; 2 000; -23 500; -21 438,5; -4 287,71
6            6; 2 000; -23 500; -21 432,5; -3 572,09
42           7; 2 000; -23 500; -21 390,5; -3 055,79
```

```
Očištěná znaková čísla: ['-23.5e3', '-.32', '42.777', '2000', '19', '6', '42']
```

Tisk vrácených výsledků

```
print(vysledky)
počet: 7; maximum: 2 000; minimum: -23 500; součet: -21 390,5; průměr: -3 055,79
```

Výstupní soubor `cisla_text.csv` s desetinnými čárkami a se středníky

```
-23 500;-0,32;42,777;
2 000;19;6;42;
```

Kontrola textového vstupu čísel v modulu `numpy`

Modul `numpy` je orientován na čísla, je velmi rozsáhlý a umožňuje mj. hromadné ovládání *číselných polí* (arrays). Při kontrole vstupního textu se postupuje jinak než v příkladu se stavovým strojem.

Modul `numpy` obsahuje metodu `numpy.genfromtxt()`, která čte textový soubor a rozdělí řádek do podřetězců oddělených čárkami, převede je na zlomková čísla typu `float`, přičemž za chybné položky dosazuje symbol **nan** (not a number). Z čísel pak vytvoří *pole* (typu `numpy.ndarray`).

Metoda `genfromtxt()` obecně z každého řádku textu (ukončeného znakem `'\n'`) vytváří číselné pole. Řádky ovšem musí obsahovat stejný počet prvků, jinak hlásí chybu. Náš příklad převzatý z kapitoly o stavovém stroji je vyhodnocen jako dvojrozměrné pole se dvěma řádky a čtyřmi sloupci. Do něj byly zařazeny hodnoty `nan` místo chybějících nebo chybných podřetězců. Jeden řetězec je prázdný – za čárkou v prvním řádku, druhý je chybný podřetězec `*19` v druhém řádku.

```
import numpy as np

with open("cisla.txt", "w") as file:
    file.write(f"""
-23.5e3, -.32, 42.777,
2_0_0_0, *19, 6 ,42
""")

with open("cisla.txt", "r") as file:
    array = np.genfromtxt(file, delimiter=",")
print(array)

[[ -2.3500e+04 -3.2000e-01  4.2777e+01      nan]
 [  2.0000e+03      nan  6.0000e+00  4.2000e+01]]

max = array.max(initial=float("-inf"))
min = array.min(initial=float("inf"))
sum = array.sum(initial=0)
nbr = array.size
print("maximum =", max, "minimum =", min, "průměr =", sum/nbr)

maximum = nan minimum = nan průměr = nan
```

Funkce `a.max()`, `a.min()`, `a.sum()` vrátily místo čísla symbol `nan`.

Chceme-li z pole získat nějaké informace, můžeme ve volání metody označit hodnoty `nan` jako chybějící a nahradit je náhradní (výplňkovou) hodnotou, například nulou.

```
with open("cisla.txt", "w") as file:
    file.write(f"""
-23.5e3, -.32, 42.777 ,
2_0_0_0, *19, 6 ,42""")

with open("cisla.txt", "r") as file:
    array = np.genfromtxt(file, delimiter=",",
                          missing_values="nan", filling_values=0)

print(array)

[[-2.3500e+04 -3.2000e-01  4.2777e+01  0.0000e+00]
 [ 2.0000e+03  0.0000e+00  6.0000e+00  4.2000e+01]]

max = array.max(initial=float("-inf"))
min = array.min(initial=float("inf"))
sum = array.sum(initial=0)
nbr = array.size
print(f"počet: {nbr}, maximum: {max}, minimum: {min}, "
      f"součet: {sum}, průměr: {sum/nbr}")
```

```
počet: 8, maximum: 2000, minimum: -23500, součet: -21409.5, průměr: -2676.19
```

Pro srovnání je zde výsledek z posledního příkladu na stavový stroj (s českou lokalizací):

```
Očištěná znaková čísla: ['-23.5e3', '-.32', '42.777', '2000', '19', '6', '42']
```

```
počet: 7; maximum: 2 000; minimum: -23 500; součet: -21 390,5; průměr: -3 055,79
```

Poznámka: Způsob kontroly textu pomocí modulu `numpy` neumožňuje hlášení chybných znaků, ani výpočet průběžných hodnot maxima, minima, součtu a průměru. Má ale výhodu ve stručnosti.

Databáze v Pythonu

Databáze je poměrně složitý systém sloužící k uchování velkého množství dat a manipulaci s nimi. Existují dva rozdílné typy databází: *relační* a *nerelační*.

Nerelační druhy databází se datují už od 1960. let a od roku 2009 se obecně nazývají NoSQL (Not only SQL). Zatímco relační databáze operují s tabulkami složenými z řádků a sloupců, ostatní operují s jinými prostředky: dvojice klíč-hodnota, dokumenty, sloupce, grafy. Nebudeme se jimi zabývat.

Relační databáze se datují asi od roku 1979 a ovládají se nejčastěji prostřednictvím programovacího jazyka **SQL** (Structured Query Language). Ačkoliv od roku 1986 probíhá jeho standardizace, jazyk SQL se u různých dodavatelů databázových systémů se od standardu liší, někdy méně, někdy více. V jazyku Python se ustálilo používání hlavních relačních databází: **SQLite**, **MySQL** a **PostgreSQL**.

Tyto databázové systémy si ukážeme na jednoduchých ukázkách. Ukázky předpokládají znalost pojmů a příkazů SQL aspoň do té míry, jak jsou v ukázkách použity. Nijak se nesnažíme vyložit jazyk SQL u různých databázových systémů, ani jeho odlišnosti. Jde *jen o předvedení mechanismů* pro vytvoření databáze a tabulek, pro naplnění tabulek daty a pro dotazování a zpracování výsledků.

Ukázka je z aplikace zásobování a bude pro jednoduchost používat jen dvě tabulky, STAVY a CENY.

V tabulce STAVY budou údaje:

ZAVOD	číslo závodu (textově)
SKLAD	číslo skladu (textově)
MATER	číslo materiálu (textově)
MNOZ	množství materiálu na skladě (zlomkové číslo)

V tabulce CENY budou údaje:

MATER	číslo materiálu (textově)
CENAJ	cena za jednotku materiálu (zlomkové číslo)
NAZEV	označení materiálu (textově)

Tyto tabulky naplníme odpovídajícími údaji a pak, propojením obou tabulek budeme zjišťovat, jaká je celková cena každého materiálu na skladě (vynásobením jednotkové ceny a množství na skladě a zaokrouhlením).

Databáze SQLite

Tento databázový systém je původně určen pro použití v jazyku C, ale je přizpůsoben pro Python pod názvem **sqlite3**, což je také jméno *standardního modulu*. Použití tohoto systému je velmi jednoduché. Na rozdíl od ostatních systémů nevyžaduje připojení k serveru (řídícímu programu databáze). Data lze umístit do diskového souboru nebo dokonce do operační paměti.

```
import sqlite3
```

Vytvoření databázových tabulek

Připojíme zvolené jméno databáze a získáme objekt typu `sqlite3.Connection`.

```
connection = sqlite3.connect('sqlite3_zasoby.db')
```

Nebo připojíme operační paměť.

```
connection = sqlite3.connect(':memory:')
```

K dalším příkazům vytvoříme objekt typu `Cursor`.

```
cursor = connection.cursor()
```

Ted' už můžeme vytvořit tabulky a naplnit je daty. Metoda **execute()** provede jeden SQL příkaz. Vytvoříme dvě tabulky: STAVY a CENY. U tabulky STAVY určujeme datové typy sloupců, u tabulky CENY typy vynecháváme. Databáze si typy dat určí, až budeme sloupce naplňovat daty.

```
cursor.execute("""
CREATE TABLE stavy
(zavod TEXT, sklad TEXT, mater TEXT, mnoz REAL)
""")
cursor.execute("CREATE TABLE ceny (mater, cenaj, nazev)")
```

Tabulku STAVY naplníme jednotlivými SQL příkazy INSERT a potvrdíme příkazem COMMIT. Typ dat bude určen typem zadané hodnoty pro sloupec. Například '01' má typ TEXT, 100.05 má typ REAL.

```
cursor.execute("INSERT INTO stavy VALUES ('01','01','40001',0)")
cursor.execute("INSERT INTO stavy VALUES ('01','02','40002',100.05)")
cursor.execute("INSERT INTO stavy VALUES ('02','01','40003',100.05)")
cursor.execute("INSERT INTO stavy VALUES ('02','02','40004',100.05)")
cursor.execute("COMMIT")
```

Databáze `sqlite3` rozeznává jen typy NULL, INTEGER, REAL, TEXT, BLOB, které odpovídají v Pythonu typům None, int, float, str, bytes.

Druhou tabulku CENY naplníme příkazem **executemany()**, který naplní několik řádků do tabulky najednou. Zde jsou typy dat u sloupců vynechány. Hodnoty sloupců v SQL příkazu INSERT jsou označeny *otazníky*, na jejichž místo se dostanou hodnoty zadané za příkazem INSERT v *seznamu* odděleném čárkou. Prvky seznamu jsou *n-tice* (trojice) hodnot sloupců, které jsou uvedeny ve správném pořadí.

```
cursor.executemany("INSERT INTO ceny (mater, cenaj, nazev) VALUES (?, ?, ?)",
[('40001',1.00,'Materiál 1'),
 ('40002',2.01,'Materiál 2'),
 ('40003',3.01,'Materiál 3'),
 ('40004',4.01,'Mater')])
```

```
connection.commit() # alternativa k SQL příkazu COMMIT
```

Když máme tabulky naplněné daty, můžeme se na ně dotazovat. Spuštění SQL příkazu `SELECT` vytvoří *výslednou tabulku (result set)*, kterou pak můžeme zpracovat různými způsoby do viditelné podoby.

Dotaz na data

```
text_dotazu = '''
SELECT zavod, sklad, S.mater, nazev, cenaj, mnoz, round(cenaj*mnoz, 2)
FROM stavy as S
JOIN ceny as C on S.mater = C.mater
ORDER BY mnoz'''
cursor.execute(text_dotazu)
```

Výsledná tabulka je objekt typu `sqlite3.Cursor`.

Můžeme ji zpracovat například tak, že vytiskneme jednotlivé záznamy (n-tice) v cyklu s použitím metody **`fetchone()`**.

```
zaznam = cursor.fetchone() # přečtu první záznam výsledné tabulky
while zaznam:
    for sloupec in range(len(zaznam)):
        if sloupec != len(zaznam)-1:
            print(f'{zaznam[sloupec]},', end='')
        else:
            print(f'{zaznam[sloupec]}', end='') # poslední sloupec bez čárky
    print()
    zaznam = cursor.fetchone() # přečtu další záznam výsledné tabulky

01,01,40001,Materiál 1,1.0,0.0,0.0
01,02,40002,Materiál 2,2.01,100.05,201.1
02,01,40003,Materiál 3,3.01,100.05,301.15
02,02,40004,Mater,4.01,100.05,401.2
```

Jiný způsob je ten, že ji proměníme na seznam n-tic metodou **`fetchall()`**. Předtím ovšem musíme *provést dotaz znovu*, protože výsledná tabulka byla předchozím zpracováním vyčerpána.

```
cursor.execute(text_dotazu)

result_set = cursor.fetchall()
print(result_set)

[('01', '01', '40001', 'Materiál 1', 1.0, 0.0, 0.0), ('01', '02', '40002',
'Materiál 2', 2.01, 100.05, 201.1), ('02', '01', '40003', 'Materiál 3', 3.01,
100.05, 301.15), ('02', '02', '40004', 'Mater', 4.01, 100.05, 401.2)]
```

V tomto seznamu každá n-tice odpovídá jednomu záznamu výsledné tabulky.

Zpracování výsledné tabulky pomocí knihovny Pandas

Použijeme výslednou tabulku `result_set` ve formě seznamu n-tic získaného metodou `fetchall()`. Můžeme ji s výhodou zpracovat různými metodami knihovny *Pandas*.

Knihovnu *Pandas* instalujeme z příkazového řádku příkazem

```
pip3.9 install pandas
```

Pak importujeme modul **pandas**.

```
import pandas as pd
```

Ve výstupu z dotazu můžeme získat nadpisy (hlavičky) pro jednotlivé sloupce. Získáme je z objektu `cursor.description`, který pro každý řádek výsledné tabulky obsahuje pracovní n-tici, jejíž nultý prvek je jméno sloupce:

```
headings = [col_name[0].upper() for col_name in cursor.description]
```

s výsledkem `['ZAVOD', 'SKLAD', 'MATER', 'NAZEV', 'CENAJ', 'MNOZ', 'CELKEM']`

Kdybychom chtěli hlavičky jiného tvaru než jsou jména sloupců tabulek, můžeme je definovat jako seznam textů v pořadí odpovídajících sloupců dat.

```
headings = ["ZAVOD", "SKLAD", "MATERIÁL", "NÁZEV", "CENA/J", "MNOŽSTVÍ",  
"CELKEM"]
```

Definujeme instanci třídy **DataFrame** (datový rámeček) pomocí jejího konstruktoru s parametrem `result_set`, tj. s výslednou SQL tabulkou.

```
df = pd.DataFrame(result_set, columns=headings) # objekt třídy DataFrame
```

Chybí-li parametr `columns`, budou hlavičkami *pořadová čísla* sloupců.

Výstup do formátu HTML

Metoda `to_html()` transformuje datový rámeček do formátu HTML a výsledek umístí do zadaného souboru, který jsme předem vytvořili.

Parametr `index=False` potlačuje výstup pořadových čísel řádků.

```
with open("sqlite3_zasoby.html", encoding="utf-8", mode="w") as outfile:  
    print(df.to_html(index=False), file=outfile)
```

```
<table border="1" class="dataframe">  
  <thead>  
    <tr style="text-align: right;">  
      <th>0</th>  
      <th>1</th>  
      ...
```

Zobrazíme-li soubor `"sqlite3_zasoby.html"` např. v prohlížeči, např. Safari (v UTF-8), dostaneme tento obrázek.

ZAVOD	SKLAD	MATERIÁL	NÁZEV	CENA/J	MNOŽSTVÍ	CELKEM
01	01	40001	Materiál 1	1.00	0.00	0.00
01	02	40002	Materiál 2	2.01	100.05	201.10
02	01	40003	Materiál 3	3.01	100.05	301.15
02	02	40004	Mater	4.01	100.05	401.20

Výstup do formátu Excel

Metoda `to_excel()` transformuje datový rámec do excelového formátu `xlsx` a výsledek umístí do zadaného souboru.

```
df.to_excel("sqlite3_zasoby.xlsx")
```

Zobrazíme-li soubor "sqlite3_zasoby.xlsx" v programu LibreOffice, dostaneme po malé úpravě tento obrázek.

	A	B	C	D	E	F	G	H
1		ZAVOD	SKLAD	MATERIÁL	NÁZEV	CENA/J	MNOŽSTVÍ	CELKEM
2	0	01	01	40001	Materiál 1	1	0	0
3	1	01	02	40002	Materiál 2	2,01	100,05	201,1
4	2	02	01	40003	Materiál 3	3,01	100,05	301,15
5	3	02	02	40004	Mater	4,01	100,05	401,2

Poznámka: V případě, že se nepodaří metodu provést, můžeme instalovat modul **openpyxl** příkazem

```
pip3.9 install openpyxl
```

Výstup do formátu CSV

Metoda `to_csv()` transformuje datový rámec do textu ve formátu *CSV (Comma Separated Values)*. Parametr `line_terminator='\n'` je zadán kvůli tomu, že bez něj by v systému *Windows* byl symbol nového řádku `\r\n`, což by *pandas* vyhodnotil jako dva nové řádky.

```
text_csv = df.to_csv(line_terminator='\n')
print(text_csv)
```

```
,ZAVOD,SKLAD,MATERIÁL,NÁZEV,CENA/J,MNOŽSTVÍ,CELKEM
0,01,01,40001,Materiál 1,1.0,0.0,0.0
1,01,02,40002,Materiál 2,2.01,100.05,201.1
2,02,01,40003,Materiál 3,3.01,100.05,301.15
3,02,02,40004,Mater,4.01,100.05,401.2
```

Když tento text zapíšeme do souboru a otevřeme jej např. v programu *LibreOffice*, dostaneme podobný výsledek jako z metody `to_excel()`, kde jsou ale hodnoty ve sloupcích jinak formátované a zarovnané.

```
with open("sqlite3_zasoby.csv", encoding="utf-8", mode="w") as outfile:
    print(text_csv, file=outfile)
```

	A	B	C	D	E	F	G	H
1		ZAVOD	SKLAD	MATERIÁL	NÁZEV	CENA/J	MNOŽSTVÍ	CELKEM
2	0	1	1	40001	Materiál 1	1.0	0.0	0.0
3	1	1	2	40002	Materiál 2	2.01	100.05	201.1
4	2	2	1	40003	Materiál 3	3.01	100.05	301.15
5	3	2	2	40004	Mater	4.01	100.05	401.2

Výstup na konzolu

Metoda `to_string()` vrátí formátovaný text výsledné tabulky.

Parametr `col_space` určuje minimální šířky výstupu jednotlivých sloupců. V příkladu jsou nastaveny tak, že mezi tiskovými sloupci (i s hlavičkami) jsou dvě mezery. Když parametr nezadáme, tiskové sloupce budou odděleny jednou mezerou.

```
formatted_text = df.to_string(col_space=[5, 6, 9, 11, 7, 9, 7],
                              index=False, justify="right")
```

ZAVOD	SKLAD	MATERIÁL	NÁZEV	CENA/J	MNOŽSTVÍ	CELKEM
01	01	40001	Materiál 1	1.00	0.00	0.00
01	02	40002	Materiál 2	2.01	100.05	201.10
02	01	40003	Materiál 3	3.01	100.05	301.15
02	02	40004	Mater	4.01	100.05	401.20

Šířka sloupce pro výstup je dána šířkou nejdelší hodnoty všech řádků. Všimněme si, že hodnoty jsou v rámci šířky sloupce zarovnány vpravo bez ohledu na datový typ. Většinou ovšem bývá žádoucí, aby textové hodnoty byly zarovnány vlevo.

Parametr `index=False` znamená potlačení tisku pořadových čísel řádků. Parametr `justify` se týká jen *zarovnání nadpisů*, může být "right", "left" nebo "center". Není-li zadán, volí se "right". V tom případě je konec textu hlavičky zarovnán nad koncem datového sloupce. Zadáme-li hodnotu "left", má text hlavičky začínat nad začátkem datového sloupce. V našem případě jsou hlavičky číselných sloupců posunuté o jedno místo doprava proti správné pozici nad levým krajem datového sloupce.

ZAVOD	SKLAD	MATERIÁL	NÁZEV	CENA/J	MNOŽSTVÍ	CELKEM
01	01	40001	Materiál 1	1.00	0.00	0.00
01	02	40002	Materiál 2	2.01	100.05	201.10
02	01	40003	Materiál 3	3.01	100.05	301.15
02	02	40004	Mater	4.01	100.05	401.20

Metoda `pd.read_sql_query()` je alternativou k použití konstruktoru třídy `pd.DataFrame`.

Jako argument přijímá text SQL příkazu `SELECT` a objekt spojení s databází a vytvoří objekt typu `DataFrame`. Na něj pak použijeme metody `to_string()`, `to_csv()`, `to_html()` atd.

```
df = pd.read_sql_query(''
SELECT zavod, sklad, S.mater, nazev, cenaj, mnoz, round(cenaj*mnoz, 2) as celkem
FROM stavy as S
JOIN ceny as C on S.mater = C.mater
ORDER BY mnoz'', connection)
print(df.to_string())
```

	zavod	sklad	mater	nazev	cenaj	mnoz	celkem
0	01	01	40001	Materiál 1	1.00	0.00	0.00
1	01	02	40002	Materiál 2	2.01	100.05	201.10
2	02	01	40003	Materiál 3	3.01	100.05	301.15
3	02	02	40004	Mater	4.01	100.05	401.20

Názvy sloupců jsou převzaty z definice dotazu.

Zápis dat do SQL tabulky z CSV souboru

Máme-li k dispozici *csv* soubor, můžeme s použitím metody `csv.reader()` z modulu **CSV** číst data z tohoto souboru a získané výsledky zapisovat do SQL tabulky *shodné struktury*:

```
CREATE TABLE ceny (mater, cenaj, nazev)
```

Máme-li například připravený soubor `sqlite3_ceny.csv`

```
"40001",1.0,"Materiál 1"  
"40002",2.01,"Materiál 2"  
"40003",3.01,"Materiál 3"  
"40004",4.01,"Mater"
```

který přesně odpovídá struktuře SQL tabulky `ceny`, můžeme jeho data vložit do tabulky `ceny` takto:

```
with open("sqlite3_ceny.csv") as infile:  
    rows = csv.reader(infile, quoting=csv.QUOTE_NONNUMERIC)  
    for row in rows: # pro každý řádek souboru  
        cursor.execute("INSERT INTO ceny VALUES (?, ?, ?)", row)  
        connection.commit()  
# zkontroluji obsah tabulky  
cursor.execute("SELECT * FROM ceny")  
print(cursor.fetchall())
```

s výsledkem

```
[('40001', 1.0, 'Materiál 1'), ('40002', 2.01, 'Materiál 2'), ('40003', 3.01,  
'Materiál 3'), ('40004', 4.01, 'Mater')]
```

Soubory ve tvaru *csv* vznikají nejrůznějšími cestami. Běžným způsobem je vznik z tabulky **DataFrame** modulu **pandas**. Můžeme si to ukázat na obráceném postupu, když vytvoříme z SQL tabulky *csv* soubor, který vznikne z metody `df.to_csv()` bez argumentů.

```
df = pd.read_sql_query('''  
SELECT mater, cenaj, nazev  
FROM ceny''', connection)  
  
with open("sqlite3_ceny.csv", encoding="utf-8", mode="w") as outfile:  
    csv_text = df.to_csv() # varianta bez argumentů  
    outfile.write(csv_text)  
    print(csv_text)  
  
,mater,cenaj,nazev  
0,40001,1.0,Materiál 1  
1,40002,2.01,Materiál 2  
2,40003,3.01,Materiál 3  
3,40004,4.01,Mater
```

I tento soubor můžeme zpětně zapsat do SQL tabulky, ačkoliv má navíc záhlaví sloupců i pořadové číslo, a nemá uvozovky u textových dat.

```
with open("sqlite3_ceny.csv") as infile:
    csv_reader = csv.reader(infile) # seznam řádků souboru
    next(csv_reader) # přeskočím hlavičky
    for row in csv_reader: # pro každý řádek souboru
        row = row[1:] # vynechám číslo sloupce
        row[1] = float(row[1]) # druhý prvek řádku (cenaj) bude typu float
        cursor.execute("""INSERT INTO ceny (mater, cenaj, nazev)
                        VALUES (?, ?, ?)""", row)
connection.commit()
```

Uzavření přístupu k databázi

Když už nepotřebujeme výslednou tabulku (result set) k dalším dotazům, můžeme uzavřít `cursor` i `connection`.

```
cursor.close()
connection.close()
```

Databáze MySQL

Příprava programovacího prostředí

Nejdříve je nutné obstarat samotný program databáze, který lze stáhnout ze stránky <https://dev.mysql.com> pro příslušný operační systém a nainstalovat jej podle návodu v dokumentaci. Při instalaci se zadává *heslo* pro uživatele `root`. V našem příkladu bylo zadáno (slabé) heslo 012345678.

Pak je nutné stáhnout tzv. *konektor* ze stránky <https://dev.mysql.com/downloads/connector/python/>. Zde vybereme **Source code** a tam pak "Platform Independent (Architecture Independent), ZIP Archive **Python**". Konektor musíme zařadit do Pythonu pomocí programu PIP z *příkazového řádku* příslušného operačního systému:

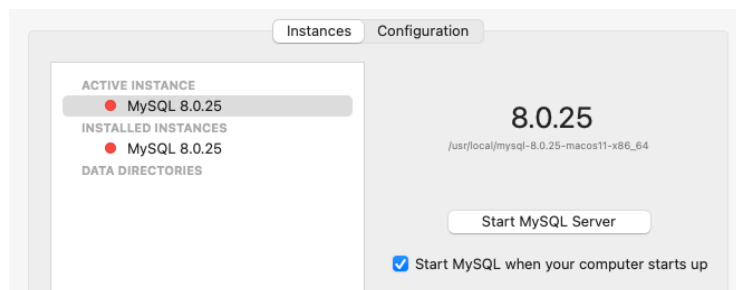
```
pip3.9 install mysql-connector-python
```

Získáme tak *modul* **mysql.connector**.

Po instalacích musíme zapnout *server*, aby byl schopen přijímat příkazy a odpovídat na ně. V různých operačních systémech se zapínání a vypínání provádí různě.

macOS

Nejjednodušší je použít předvolbu  v Předvolbách (Preference Pane).



Druhá možnost je použít příkazy v programu Terminal, k nimž je třeba zadat heslo administrátora macOS.

Zapnutí serveru:

```
$ cd /Library/LaunchDaemons  
$ sudo launchctl load -F com.oracle.oss.mysql.mysqld.plist
```

Vypnutí serveru:

```
$ cd /Library/LaunchDaemons  
$ sudo launchctl unload -F com.oracle.oss.mysql.mysqld.plist
```

Windows

V seznamu programů najdeme Nástroje pro správu Windows a otevřeme okno Služby (Services). Tam můžeme zapnout a vypnout server MySQL.

Linux

Zjištění stavu serveru

```
$ systemctl status mysql.service
```

Vypnutí serveru

```
$ sudo systemctl start mysql
```

Zapnutí serveru

```
$ sudo systemctl stop mysql
```

Po instalacích už můžeme vytvořit vlastní databázi (schema) a v něm tabulky, do nichž zapíšeme data. Načež se můžeme na data dotazovat a získané údaje zobrazovat.

Vytvoření databázových tabulek

Ukázka je ze stejné aplikace zásobování a bude používat také dvě tabulky, STAVY a CENY. Jejich údaje budou mít tentokrát přesně stanovené typy a velikosti.

V tabulce STAVY budou údaje:

ZAVOD	číslo závodu CHAR(2)
SKLAD	číslo skladu CHAR(2)
MATER	číslo materiálu CHAR(5)
MNOZ	množství materiálu na skladě DECIMAL(10,2)

V tabulce CENY budou údaje:

MATER	číslo materiálu CHAR(5)
CENAJ	cena za jednotku materiálu DECIMAL(10,2)
NAZEV	označení materiálu VARCHAR(50)

Tyto tabulky naplníme odpovídajícími údaji a pak, propojením obou tabulek budeme zjišťovat, jaká je celková cena každého materiálu na skladě (vynásobením jednotkové ceny a množství na skladě).

Poznámka: Za SQL příkazy mohou a nemusí být zapsány koncové středníky.

```
import mysql.connector
```

Zvolíme jméno schematu (v MySQL se používá místo "schema" pojem "database").

```
schema_name = "zasoby" # jméno schematu = jméno databáze
```

Provedeme první připojení k serveru MySQL, ještě bez schematu (databáze). Server (řídící program databáze) je lokální. Uživatel a heslo jsou povinné údaje.

```
connection = None
try:
    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="012345678"
    )
    print("MySQL database connection created.")
except mysql.connector.errors.DatabaseError as err:
    print(f"Error: {err}. Program is ended.")
    exit(1)
```

Připojovací objekt `connection` je instance třídy `Connection`. Z něj dále vytvoříme objekt třídy `Cursor`, který již umožňuje provádět příkazy SQL.

```
cursor = connection.cursor()
```

K umístění tabulek musíme vytvořit `SCHEMA` a doplnit je do spojení.

```
cursor.execute(f"CREATE SCHEMA {schema_name}") # "zasoby"  
connection.database = schema_name # doplníme schema do spojení
```

Ted' už můžeme vytvořit tabulky a naplnit je daty. Metoda **`execute()`** provede jeden SQL příkaz. Vytvoříme dvě tabulky: `STAVY` a `CENY`. Na rozdíl od databáze `SQLite` u sloupců definujeme typy a rozměry.

```
cursor.execute('''  
CREATE TABLE stavy  
    (zavod char(2),  
     sklad char(2),  
     mater char(5),  
     mnoz decimal(10,2))''')  
cursor.execute('''  
CREATE TABLE ceny  
    (mater char(5),  
     cenaj decimal(10,2),  
     nazev varchar(50))''')
```

Tabulku `STAVY` naplníme příkazy **`execute()`** s SQL příkazy `INSERT` pro jednotlivé řádky tabulky.

```
cursor.execute("INSERT INTO stavy VALUES ('01','01','40001',0)")  
cursor.execute("INSERT INTO stavy VALUES ('01','02','40002',100.05)")  
cursor.execute("INSERT INTO stavy VALUES ('02','01','40003',100.05)")  
cursor.execute("INSERT INTO stavy VALUES ('02','02','40004',100.05)")
```

Druhou tabulku `CENY` naplníme metodou **`executemany()`**, která naplní několik řádků do tabulky najednou v jednom SQL příkazu `INSERT`. Značky `%s` označují místa, kam patří příslušné hodnoty sloupců podle pořadí.

```
sql = "INSERT INTO ceny VALUES (%s, %s,%s)"  
val = [('40001', 1.01, 'Materiál 1'),  
       ('40002', 2.01, 'Materiál 2'),  
       ('40003', 3.01, 'Materiál 3'),  
       ('40004', 4.01, 'Materiál 4')]
```

```
cursor.executemany(sql, val)
```

Data v tabulkách jsme potvrdili příkazem **`commit()`**, jinak by se data ztratila.

Dotaz na data

```
cursor.execute('''
SELECT zavod, sklad, S.mater, nazev, cenaj, mnoz, round(cenaj*mnoz, 2)
FROM stavy as S
JOIN ceny as C on S.mater = C.mater
ORDER BY mnoz''')
```

Metodou **fetchall()** získáme všechny záznamy (řádky) výsledné tabulky z objektu typu **Cursor**.

```
result_set = cursor.fetchall()
print(result_set)
```

Výsledkem je tento seznam n-tic, tentokrát s desítkovými čísly typu **Decimal**:

```
[('01', '01', '40001', 'Materiál 1', Decimal('1.01'), Decimal('0.00'),
Decimal('0.00')), ('01', '02', '40002', 'Materiál 2', Decimal('2.01'),
Decimal('100.05'), Decimal('201.10')), ('02', '01', '40003', 'Materiál 3',
Decimal('3.01'), Decimal('100.05'), Decimal('301.15')), ('02', '02', '40004',
'Mater', Decimal('4.01'), Decimal('100.05'), Decimal('401.20'))]
```

Můžeme postupovat naprosto stejně jako v předchozím příkladu databáze SQLite, teď ale zkusíme vlastní způsob formátování výsledné tabulky.

Zpracování výsledků dotazu vlastní funkcí

Použijeme vlastní funkci **format_result_set()** z modulu **spolecne_funkce.py**, který jsme si vytvořili i pro jiné funkce.

Funkce má tři parametry:

result_set	seznam n-tic odpovídající výsledné tabulce z SQL dotazu
headings	seznam textů hlaviček
separators	počet mezer oddělujících tiskové sloupce

Funkce vrátí seznam formátovaných řádků určených pro tisk nebo jiné zpracování. Zde v MySQL je výsledná tabulka už rovnou *seznam n-tic* (ne objekt typu **Cursor** jako v SQLite).

```
from spolecne_funkce import format_result_set
```

```
list_of_lines =
    format_result_set(result_set=result_set, headings=headings, separators=2 )
```

```
for line in list_of_lines:
    print(line)
```

ZAVOD	SKLAD	MATER	NÁZEV	CENA/J	MNOŽSTVÍ	CELKEM
01	01	40001	Materiál 1	1.01	0.00	0.00
01	02	40002	Materiál 2	2.01	100.05	201.10
02	01	40003	Materiál 3	3.01	100.05	301.15
02	02	40004	Mater	4.01	100.05	401.20

Tady jsou hodnoty v textových sloupcích zarovnány vlevo, v číselných sloupcích vpravo. Pod hlavičkami jsou textové sloupce zarovnány vlevo, číselné sloupce vpravo. Mezi tiskovými sloupci jsou vynechány dvě mezery, jak je předepsáno.

Zápis dat do SQL tabulky z CSV souboru

Máme-li k dispozici *csv* soubor, můžeme s použitím metody **reader()** z *modulu* *CSV* číst data z tohoto souboru a získané výsledky zapisovat do SQL tabulky *shodné struktury*:

```
cursor.execute('''
CREATE TABLE ceny
    (mater char(5),
    cenaj decimal(10,2),
    nazev varchar(50))''')
```

Máme-li například soubor `mysql_ceny.csv`, třeba bez uvozovek u textových dat,

```
40001,1.0,Materiál 1
40002,2.01,Materiál 2
40003,3.01,Materiál 3
40004,4.01,Mater
```

který přesně kopíruje strukturu SQL tabulky *ceny*, můžeme data vložit do této tabulky takto:

```
with open("mysql_ceny.csv", mode="r") as infile:
    rows = csv.reader(infile)
    for row in rows:
        cursor.execute("INSERT INTO ceny VALUES (%s, %s, %s)", row)
    cursor.execute("SELECT * FROM ceny")
    print(cursor.fetchall())
```

s výsledkem

```
[('40001', Decimal('1.01'), 'Materiál 1'), ('40002', Decimal('2.01'), 'Materiál 2'), ('40003', Decimal('3.01'), 'Materiál 3'), ('40004', Decimal('4.01'), 'Mater')]
```

Uzavření přístupu k databázi

Když už nepotřebujeme výslednou tabulku (result set) k dalším dotazům, můžeme uzavřít `cursor` i `connection`.

```
cursor.close()
connection.close()
```

Funkce `format_result_set()`

```
def format_result_set(result_set, headings, separators=1):
    """
    Výpočet šířky sloupců pro tisk z maximálních délek hodnot ve sloupcích
    a délek hlaviček.
    Zarovnání hlaviček a zarovnání detailních řádků.
    U číselných sloupců se data i hlavičky zarovnají vpravo.
    U ostatních sloupců se zarovnají vlevo.
    U typu 'bytes' (např. b'\xc3\xa1' = 'á') se délka dat počítá
    jako délka textové reprezentace: len(repr(item)).
    """

    assert type(result_set) is list, \
        f"Parameter resultset must be a list, but is {result_set}."
    assert type(headings) is list or headings is None, \
        f"Parameter headings must be a list or None, but it is {headings}."
    assert type(separators) is int, \
        f"Column separator length must be of type int, but is {separators}."
    import decimal
    heads = headings.copy() # kopie hlaviček, aby se původní neměnily
    col_max_widths = [] # maximální délky hodnot ve sloupcích
    col_max_decpos = [] # maximální počty des. míst u typu float nebo Decimal
    sep_spaces = separators * " "

    # výpočet šířky výstupních sloupců – max. délka hodnot převedených na text
    for col in range(len(result_set[0])): # počet sloupců
        max_item_len = 0
        max_decpos = 0
        for row in range(len(result_set)): # beru sloupec ze všech řádků seznamu
            if type(result_set[row][col]) == int \
                or type(result_set[row][col]) is float \
                or type(result_set[row][col]) is decimal.Decimal:
                item = str(result_set[row][col]) # čísla převedu na text
            else:
                item = result_set[row][col]
            # počítám maximální délku hodnot
            item_len = len(item)
            if type(item) == bytes:
                item_len = len(repr(item))
            if item_len > max_item_len:
                max_item_len = item_len
            # počítám maximální počet desetinných míst
            if type(result_set[row][col]) is float \
                or type(result_set[row][col]) is decimal.Decimal:
                dec_pos = len(str(item)[item.find('.')+1:])
                if dec_pos > max_decpos:
                    max_decpos = dec_pos
            # přidávám do seznamu maximálních počtů desetinných míst
            col_max_decpos.append(max_decpos)
            # přidávám do seznamu maximálních délek hodnot
            col_max_widths.append(max_item_len)
        if heads:
            num_hdg = len(heads)
            num_col = len(result_set[0])
            if num_hdg < num_col:
                # počet sloupcových hlaviček je menší než počet sloupců dat
```

```

        for n in range(num_col - num_hdg):
            heads.append('*') # doplním hvězdičky místo chybějících hlaviček
    if num_hdg > num_col:
        # počet sloupcových hlaviček je větší než počet sloupců dat
        heads = heads[:num_col] # zkrátím hlavičky od konce na počet sloupců dat
    for col in range(len(result_set[0])):
        if len(heads[col]) > col_max_widths[col]: # hlavička delší než sloupec dat?
            # spočítám maximum délek hlaviček a sloupců
            col_max_widths[col] = len(heads[col])
        if len(heads[col]) < col_max_widths[col]: # hlavička kratší než sloupec dat?
            if type(result_set[0][col]) is int or type(result_set[0][col]) is float:
                # číselný sloupec - zarovná hlavičku vpravo
                heads[col] = (col_max_widths[col] - len(heads[col])) * " " \
                    + heads[col]
            else: # nečíselný sloupec - zarovná hlavičku vlevo
                heads[col] += (col_max_widths[col] - len(heads[col])) * " "

# výstup hlaviček do řádku
out_result_set = [] # výstupní seznam řádků
line = ''
col = 0
if heads:
    for heading in heads:
        line += f"{heading}" + sep_spaces
        col += 1
    out_result_set.append(line)

# formátování tabulky v řádcích
for a_tuple in result_set:
    line = ''
    col = 0
    for item in a_tuple:
        if type(item) is int or type(item) is float or type(item) is decimal.Decimal:
            item = str(item) # převod na text
            dec_pos = len(str(item)[item.find('.') + 1:]) # počet des. míst
            if dec_pos < col_max_decpos[col]:
                zeros = (col_max_decpos[col] - dec_pos) * "0" # rozdíl od max des. míst
                item += zeros # doplním koncové nuly
            item = (col_max_widths[col] - len(item)) * " " + item # dopln mezerami zleva
            # přidat další hodnotu sloupce oddělenou sep_spaces mezerami
            line += f"{item}" + sep_spaces
        elif type(item) == bytes:
            bspaces = (col_max_widths[col] - len(repr(item))) * " "
            # přidat další hodnotu sloupce doplněnou mezerami zprava
            line += f"{item}" + sep_spaces + bspaces
        elif type(item) == str:
            item += (col_max_widths[col] - len(item)) * " " # dopln mezerami zprava
            line += f"{item}" + sep_spaces # přidat další hodnotu sloupce
        else:
            line += f"{item}" + sep_spaces # přidat další hodnotu sloupce
        col += 1
    out_result_set.append(line)
return out_result_set

```

Databáze PostgreSQL

Příprava programovacího prostředí

Nejdříve obstaráme program databáze ze stránky <https://www.postgresql.org/download/> pro příslušný operační systém a nainstalovat jej podle návodu v dokumentaci. Při instalaci se zadává *heslo* pro uživatele `postgres`. V našem příkladu bylo zadáno (slabé) heslo 012345678.

Dále je zapotřebí získat modul `psycopg2`, který umožňuje používat databázi PostgreSQL v jazyku Python. Modul zařadíme do Pythonu pomocí programu PIP z *příkazového řádku* příslušného operačního systému:

```
$ pip3.9 install psycopg2-binary
```

Po instalacích musíme zapnout server, aby byl schopen přijímat příkazy a odpovídat na ně. V různých operačních systémech se zapínání a vypínání provádí různě.

K *zapínání a vypínání serveru* použijeme program `pgAdmin`, který v systému **macOS** je obsažen v instalovaném adresáři, a který zvlášť stáhneme v systému **Windows** (ze stránky <https://www.postgresql.org/ftp/pgadmin/>) a instalujeme. Systémy Linux vynecháváme.

Po instalacích a zapnutí serveru už můžeme vytvořit vlastní databázi (schema) a v něm tabulky, do nichž zapíšeme data. Načež se můžeme na data dotazovat a získané údaje zobrazovat. Další postup je téměř stejný jako v případě MySQL.

Vytvoření databázových tabulek

Ukázka je ze stejné aplikace zásobování a bude používat také dvě tabulky, STAVY a CENY. Jejich údaje budou mít tentokrát přesně stanovené typy a velikosti.

V tabulce STAVY budou údaje:

ZAVOD	číslo závodu CHAR(2)
SKLAD	číslo skladu CHAR(2)
MATER	číslo materiálu CHAR(5)
MNOZ	množství materiálu na skladě DECIMAL(10,2)

V tabulce CENY budou údaje:

MATER	číslo materiálu CHAR(5)
CENAJ	cena za jednotku materiálu DECIMAL(10,2)
NAZEV	označení materiálu VARCHAR(50)

Tyto tabulky naplníme odpovídajícími údaji a pak, propojením obou tabulek budeme zjišťovat, jaká je celková cena každého materiálu na skladě (vynásobením jednotkové ceny a množství na skladě).

Poznámka: Koncové středníky za SQL příkazy mohou a nemusí být zapsány.

```
import psycopg2
```

Zvolíme jméno schematu.

```
schema_name = "zasoby" # jméno schematu = jméno databáze
```

Provedeme první připojení k serveru. Server (řídící program databáze) je lokální. Uživatel a heslo jsou povinné údaje.

```
connection = None
```

```
try:
    connection = psycopg2.connect(
        host="localhost",
        user="vzupka",
        database="postgres",
        password="012345678"
    )
    print("PostgreSQL database connection created.")
except Exception as err:
    print(f"Error: {err}. Program is ending. ")
    exit(1)
```

Připojovací objekt `connection` je instance třídy `Connection`. Z něj dále vytvoříme objekt třídy `Cursor`, který již umožňuje provádět příkazy SQL.

```
cursor = connection.cursor()
```

K umístění tabulek musíme vytvořit SCHEMA.

```
cursor.execute(f"CREATE SCHEMA {schema_name}") # "zasoby"
```

Ted' už můžeme vytvořit tabulky a naplnit je daty. Metoda `execute()` provede jeden SQL příkaz. Vytvoříme dvě tabulky: STAVY a CENY. Na rozdíl od databáze SQLite u sloupců definujeme typy a rozměry.

```
cursor.execute('''
CREATE TABLE stavy
    (zavod char(2),
     sklad char(2),
     mater char(5),
     mnoz decimal(10,2))''')
cursor.execute('''
CREATE TABLE ceny
    (mater char(5),
     cenaj decimal(10,2),
     nazev varchar(50))''')
```

Tabulku STAVY naplníme metodou `execute()` s SQL příkazy INSERT pro jednotlivé řádky tabulky.

```
cursor.execute("INSERT INTO stavy VALUES ('01','01','40001',0)")
cursor.execute("INSERT INTO stavy VALUES ('01','02','40002',100.05)")
cursor.execute("INSERT INTO stavy VALUES ('02','01','40003',100.05)")
cursor.execute("INSERT INTO stavy VALUES ('02','02','40004',100.05)")
```


Druhou tabulku CENY naplníme metodou **executemany()**, která naplní několik řádků do tabulky najednou v jednom SQL příkazu INSERT. Značky **%s** označují místa, kam patří příslušné hodnoty sloupců podle pořadí.

```
sql = "INSERT INTO ceny VALUES (%s, %s,%s)"
val = [('40001', 1.01, 'Materiál 1'),
      ('40002', 2.01, 'Materiál 2'),
      ('40003', 3.01, 'Materiál 3'),
      ('40004', 4.01, 'Materiál 4')]

cursor.executemany(sql, val)
connection.commit()
```

Data v tabulkách jsme potvrdili příkazem `connection.commit()`, jinak by se data ztratila.

Dotaz na data

```
cursor.execute('''
SELECT zavod, sklad, S.mater, nazev, cenaj, mnoz, round(cenaj*mnoz, 2)
FROM stavy as S
JOIN ceny as C on S.mater = C.mater
ORDER BY mnoz''')
```

Metodou `fetchall()` získáme všechny záznamy (řádky) výsledné tabulky z objektu typu `Cursor`.

```
result_set = cursor.fetchall()
print(result_set)
```

Výsledkem je tento seznam n-tic s desítkovými čísly typu `Decimal`:

```
[('01', '01', '40001', 'Materiál 1', Decimal('1.01'), Decimal('0.00'),
Decimal('0.00')), ('01', '02', '40002', 'Materiál 2', Decimal('2.01'),
Decimal('100.05'), Decimal('201.10')), ('02', '01', '40003', 'Materiál 3',
Decimal('3.01'), Decimal('100.05'), Decimal('301.15')), ('02', '02', '40004',
'Mater', Decimal('4.01'), Decimal('100.05'), Decimal('401.20'))]
```

Zpracování výsledků dotazu

Můžeme postupovat naprosto stejně jako v předchozím příkladu databáze SQLite nebo MySQL.

Zápis dat do SQL tabulky z CSV souboru

Máme-li například soubor `pgsql_ceny.csv`, třeba bez uvozovek u textových dat,

```
40001,1.0,Materiál 1
40002,2.01,Materiál 2
40003,3.01,Materiál 3
40004,4.01,Mater
```

který přesně odpovídá struktuře SQL tabulky `ceny`, můžeme data vložit do této tabulky s použitím specifické metody `copy_from()`:

```
with open("pgsql_ceny.csv", mode="r") as infile:
    cursor.copy_from(infile, "ceny", sep=",")
    cursor.execute("SELECT * FROM ceny")
    print(cursor.fetchall())
```

s výsledkem

```
[('40001', Decimal('1.01'), 'Materiál 1'), ('40002', Decimal('2.01'), 'Materiál 2'), ('40003', Decimal('3.01'), 'Materiál 3'), ('40004', Decimal('4.01'), 'Materiál 4')]
```

Uzavření přístupu k databázi

Když už nepotřebujeme výslednou tabulku (result set) k dalším dotazům, můžeme uzavřít `cursor` i `connection`.

```
cursor.close()
connection.close()
```